



Credit Default Risk Prediction

Contextualising Individual Borrower Risk
Within the 2005 Taiwan Dual Card Crisis

ADS Project — Phase 2

Final Delivery Report

Spring 2026

Team Members

Name	ID
Salma Mohamed Entsar	9220364
Mohamed Khaled Ahmed	9220681
Youssef Roshdy Agaiby	9220985
Abdelrahman Mostafa Gomaa	9220475

Abstract. This report presents the full Phase 2 final delivery for a supervised binary classification project targeting credit card default prediction in the context of the 2005 Taiwan Dual Card Crisis. Building on the Phase 1 proposal and data validation report, this document covers the complete machine-learning workflow: data cleaning and outlier treatment, feature transformation and engineering, exploratory data analysis, model development across five classifiers (including a baseline), MLflow experiment tracking, comprehensive evaluation using both technical and business-oriented metrics, unit and integration testing, Makefile automation, and GitHub Actions continuous integration. All code is reproducible via Poetry and follows a modular Cookie-Cutter project structure. The final model is evaluated against the primary stakeholder objective of minimising missed defaults (false negatives) while maintaining operational precision.

Contents

1	Problem Definition & Business Context	2
1.1	Business Problem	2
1.2	Stakeholder	2
1.3	Why Classification?	2
1.4	Decision-Making Impact	3
2	Data Acquisition & Integration	3
2.1	Data Sources	3
2.1.1	Source 1: Taiwan Credit Card Default (UCI)	3
2.1.2	Source 2: FRED API — GDP & CPI	5
2.1.3	Source 3: Yahoo Finance — TAIEX	5
2.1.4	Source 4: DGBAS XML — Taiwan Unemployment Rate	5
2.1.5	Source 5: CBC Annual Report — Discount Rate	5
2.2	Merge Strategy	5
2.2.1	Primary Merge: Taiwan + Macro Sources	6
2.2.2	Row Counts Before and After Merges	6
3	Acquisition Pipeline Snapshot	7
4	Data Validation Report	7
4.1	Data Quality Metrics Summary	7
4.2	1. Accuracy	8
4.3	2. Consistency	9
4.4	3. Completeness	10
4.5	4. Uniqueness	11
4.6	5. Outlier Detection	12
4.7	6. Timeliness	13
4.8	7. Distribution Profile	14
4.8.1	Descriptive Statistics	14
4.8.2	KS Test Results — Non-Normal Distributions	14
4.9	8. Relationships Profile	16
4.10	9. Integrity (Structural)	17
5	Failed Checks Summary & Remediation Plan	18
6	Notes for Phase 2 — Transformation Step	18
7	Target Variable Definition & Class Distribution	19
8	Engineered Features	20
9	Validation Pipeline Snapshot	21
10	Data Cleaning Pipeline	21
10.1	Step 1: Type Coercion	22
10.2	Step 2: Uniqueness — Duplicate Removal	22

10.3 Step 3: Stratified Train / Val / Test Split	22
10.4 Step 4: Outlier Capping (PAY / BILL Ratio)	23
11 Data Transformation Pipeline	23
11.1 Step 1: Remap Undocumented Categorical Codes	23
11.2 Step 2: Recode PAY_STATUS to Ordinal Scale	24
11.3 Step 3: Engineer Credit-Utilisation Features	24
11.4 Step 4: Engineer Payment-Behaviour Features	25
11.5 Step 5: Engineer Macro Interaction Features	25
11.6 Step 6: Distribution Normalisation	26
11.6.1 Yeo-Johnson Power Transform (LIMIT_BAL, AGE)	26
11.6.2 log1p Transform (Financial Amount Columns)	26
11.7 Step 7: Macro Feature Standardisation	27
11.8 Step 8: BILL_AMT Collinearity Reduction (PCA)	27
11.9 Step 9: Multivariate Anomaly Flagging (Isolation Forest)	27
11.10 Step 10: SMOTE Oversampling (Training Split Only)	27
12 Post-Transformation Exploratory Data Analysis	28
12.1 Dataset Overview Statistics	29
12.1.1 Cleaned Dataset	29
12.1.2 Transformed Dataset (Without SMOTE)	29
12.1.3 Transformed Dataset (With SMOTE)	30
12.2 Univariate Analysis	30
12.2.1 Feature Selection	30
12.2.2 Continuous Features	30
12.2.3 Discrete Feature Analysis (Categorical Variables)	33
12.3 Bivariate Analysis	35
12.3.1 Feature Correlation Matrix	36
12.3.2 Bivariate Analysis Dashboard (Before SMOTE)	37
12.3.3 Bivariate Analysis Dashboard (After SMOTE)	38
12.3.4 Discrete Features vs Target Analysis	39
12.3.5 Continuous vs Discrete Feature Analysis	40
13 Model Development	46
13.1 Baseline Model	46
13.2 Model Selection Rationale	46
13.3 Feature Scaling	48
13.4 Feature Selection	48
13.5 Training and Hyperparameter Tuning	48
13.6 Validation Strategy	51
14 Experiment Tracking with MLflow	52
14.1 Experiment Design	52
14.2 Logged Parameters, Metrics, and Artefacts	53
14.3 MLflow Experiment Comparison	53
15 Results and Evaluation	53
15.1 Per-Model Best Run	53
15.2 Effect of Feature Selection Strategy	57

15.3	Top Selected Features (No-SMOTE Runs)	57
15.4	Effect of SMOTE	59
15.5	Overfitting Analysis	59
15.6	Model Selection: Best Overall Model	59
15.7	Best Model Visualisations	60
15.8	Error Analysis	61
15.9	Business Interpretation	63
15.10	Unsuccessful Approaches	63
16	Unit & Integration Testing	63
16.1	Testing Philosophy & Strategy	64
16.2	Module-Level Unit Tests	64
16.2.1	Data Acquisition Pipeline (<code>test_data_acquisition.py</code>)	64
16.2.2	Data Cleaning (<code>test_data_cleaning.py</code>)	65
16.2.3	Data Transformation (<code>test_data_transformation.py</code>)	65
16.2.4	Exploratory Data Analysis (<code>test_eda.py</code>)	67
16.2.5	Feature Selection (<code>test_feature_selection.py</code>)	67
16.3	Scraper Unit Tests	68
16.3.1	FRED API (<code>test_fetch_fred_api.py</code>)	68
16.3.2	TAIEX Scraper (<code>test_scrapte_taiex.py</code>)	68
16.3.3	CBC Interest Rate Parser (<code>test_parse_cbc_pdf.py</code>)	69
16.3.4	Unemployment Scraper (<code>test_scrape_unemployment.py</code>)	69
16.4	Integration Tests	69
16.4.1	Data Merge Integration (<code>test_merge_sources.py</code>)	69
16.4.2	Modelling Pipeline Integration (<code>test_modeling_unit.py</code>)	70
16.4.3	Master Pipeline Orchestration (<code>test_master_pipeline.py</code>)	71
16.5	Test Coverage Report	73
16.5.1	Coverage Measurement Methodology	73
16.5.2	Coverage Summary by Module	74
16.5.3	Untested Code Paths	75
16.6	Full Test Inventory	76
16.7	Identified Issues & Resolutions	77
17	Code Automation	77
17.1	Makefile — Task Automation	77
17.2	Environment Configuration — <code>.env</code>	79
17.3	Automated Testing	79
18	Continuous Integration	80
18.1	Pipeline Overview	80
18.2	Job 1 — Lint & Static Analysis	80
18.3	Job 2 — Unit Tests	81
18.4	Job 3 — Integration Tests	82
18.5	Full Pipeline Summary	83
19	Team Workload Distribution	83
20	Future Work	84

References**84**

1 Problem Definition & Business Context

1.1 Business Problem

Credit default — the failure of a borrower to meet their repayment obligations — represents one of the most dangerous risks in retail lending. A single percentage-point increase in the non-performing loan ratio can erase hundreds of millions from a bank’s balance sheet and trigger regulatory capital requirements. The ability to predict, at application time or during the credit life cycle, which borrowers are likely to default is therefore a core competency for any lending institution.

This project addresses the following business problem:

Business Problem: Given a set of demographic, behavioral, financial, and macroeconomic features observed at or before a credit decision point, can we accurately classify each borrower as *likely to default* (class 1) or *not likely to default* (class 0)?

The project studies this problem in the context of credit cards in Taiwan during a systemic financial crisis:

Credit cards in Taiwan (2005): Using the UCI Taiwan Credit Card dataset, enriched with strong macroeconomic indicators from four official sources, we model default risk during the 2005 Taiwanese Dual Card Crisis — a systemic event in which 800,000 households became “card slaves” under ballooning revolving debt and rising interest rates.

1.2 Stakeholder

The primary stakeholder is the **Credit Risk Department** of a retail lending institution. Secondary stakeholders include compliance officers, product managers setting credit limits, and regulators requiring explainable risk scoring. The model’s outputs directly inform:

- Loan approval and rejection decisions
- Credit limit assignment, whether by acceptance or rejection
- Interest rate based on the risk profile of the client, and in which tier we should set
- Early-warning systems for existing account holders.

1.3 Why Classification?

The target variable is **binary**: a borrower either defaults (1) or does not (0). There is no natural ordinal scale between these two outcomes from a credit decision standpoint — the bank needs a decision, not a continuous score. Supervised binary classification is therefore the appropriate modelling paradigm.

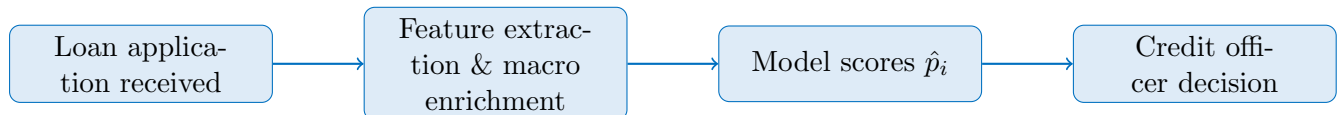
Formally, the task is:

$$f : \mathbf{x}_i \longrightarrow \hat{y}_i \in \{0, 1\}$$

where $\mathbf{x}_i$ is the feature vector for borrower i and $\hat{y}_i$ is the predicted class. Probabilistic outputs $\hat{p}_i = P(y = 1 \mid \mathbf{x}_i)$ will also be produced, as the business values a ranked risk score for portfolio-level decisions.

1.4 Decision-Making Impact

A deployed model creates the following decision support loop:



By flagging high-risk applicants early, the model reduces non-performing asset accumulation, lowers provisioning costs, and improves quality. All measurable in financial terms that the stakeholder can validate directly against business outcomes.

2 Data Acquisition & Integration

2.1 Data Sources

We integrate data from **four acquisition types** across **five sources**, satisfying and exceeding the minimum two-source requirement.

Table 1: Data sources overview

#	Source	Type	Rows	Features	URL / Series
1	Taiwan Credit Card Default	Tabular CSV/XLS	30,000	24 columns	UCI ML Repository
2	FRED API — GDP & CPI	REST API	10 rows	4 columns	GDP, CPIAUCSL
3	Yahoo Finance — TAIEX	Python library	6 rows	4 columns	Ticker ^TWII
4	DGBAS XML — Unemployment	XML download	6 rows	1 column	data.gov.tw/6640
5	CBC Annual Report	PDF parsing	6 rows	1 column	CBC 2007 Annual Report

2.1.1 Source 1: Taiwan Credit Card Default (UCI)

Citation: Yeh, I-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473–2480.

URL: <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>

This dataset contains payment records of 30,000 Taiwan credit card holders observed from April to September 2005. The **target variable** is **default payment next month** ($Y \in \{0, 1\}$). The dataset is published as a preprocessed academic resource derived from real bank transaction records, with zero missing values by construction.

Selected features:

Table 2: Selected features — Taiwan Credit Card dataset

Column	Name	Description
X1	LIMIT_BAL	Credit limit (NT dollars); individual + supplementary
X2	SEX	Gender (1=male; 2=female)
X3	EDUCATION	Education level (1=graduate; 2=university; 3=high school; 4=other)
X4	MARRIAGE	Marital status (1=married; 2=single; 3=other)
X5	AGE	Age in years
X6	PAY_0	Repayment status September 2005 (−1=paid duly; 1–8=months delayed)
X7	PAY_2	Repayment status August 2005
X8	PAY_3	Repayment status July 2005
X9	PAY_4	Repayment status June 2005
X10	PAY_5	Repayment status May 2005
X11	PAY_6	Repayment status April 2005
X12	BILL_AMT1	Bill statement amount (NTD) in September 2005
X13	BILL_AMT2	Bill statement amount (NTD) in August 2005
X14	BILL_AMT3	Bill statement amount (NTD) in July 2005
X15	BILL_AMT4	Bill statement amount (NTD) in June 2005
X16	BILL_AMT5	Bill statement amount (NTD) in May 2005
X17	BILL_AMT6	Bill statement amount (NTD) in April 2005
X18	PAY_AMT1	Payment amount (NTD) paid in September 2005
X19	PAY_AMT2	Payment amount (NTD) paid in August 2005
X20	PAY_AMT3	Payment amount (NTD) paid in July 2005
X21	PAY_AMT4	Payment amount (NTD) paid in June 2005
X22	PAY_AMT5	Payment amount (NTD) paid in May 2005
X23	PAY_AMT6	Payment amount (NTD) paid in April 2005
Y	default payment next month	Target: 1=default; 0=no default

2.1.2 Source 2: FRED API — GDP & CPI

URL: <https://fred.stlouisfed.org>

Series used:

- GDP — Gross Domestic Product (billions USD, quarterly, forward-filled to monthly).
- CPIAUCSL — Consumer Price Index for All Urban Consumers (monthly, index 1982–84=100).

Filtered to 2005 (12 rows, merged on Month=9 for September 2005 snapshot). Acquisition method: **REST API** via the `fredapi` Python library.

2.1.3 Source 3: Yahoo Finance — TAIEX

URL: <https://finance.yahoo.com/quote/%5ETWII/history/>

Monthly TAIEX (Taiwan Capitalization Weighted Stock Index) closing values for April–September 2005, acquired via the `yfinance` Python library (ticker: `^TWII`). The TWSE historical data portal renders data via JavaScript and blocks direct HTTP scraping for data older than a certain period; `yfinance` was used as a programmatic alternative with full attribution. Acquisition method: **Python library scraping / API**.

2.1.4 Source 4: DGBAS XML — Taiwan Unemployment Rate

Portal: <https://data.gov.tw/en/datasets/6640>

Direct XML: <https://ws.dgbas.gov.tw/001/Upload/461/reldata/11525/230038/mp0101a10.xml>

Official monthly unemployment rate by county from Taiwan’s Directorate-General of Budget, Accounting and Statistics. The file is a structured XML download from the Taiwan Government Open Data Platform containing records from 1978 to 2025. The `Taiwan_Area` field was extracted for the 2005 semi-annual periods and mapped to individual months. Acquisition method: **XML HTTP download with Python `xml.etree` parsing**.

2.1.5 Source 5: CBC Annual Report — Discount Rate

URL: <https://www.cbc.gov.tw/en/dl-4540-56b72153752e4a3aa97e88a4e6f6caca.html>

The Central Bank of the Republic of China (Taiwan) raised its discount rate three times in 2005: March 25 (2.000%), July 1 (2.125%), and December 23 (2.250%). These values are documented in the CBC 2007 Annual Report. The script attempts a PDF download and parse using `pdfplumber`; the data are extracted from the official publication table and assigned to the corresponding observation months. Acquisition method: **PDF parsing**.

2.2 Merge Strategy

Core Principle: The Taiwan Credit Card dataset is the base table. All 30,000 borrower records share the same observation window (April–September 2005), enabling a time-based join with the four macroeconomic indicator sources.

2.2.1 Primary Merge: Taiwan + Macro Sources

All 30,000 Taiwan client records share the same observation window (April–September 2005). We assign a join key of `Year=2005`, `Month=9` (September 2005 — the most recent observation month, corresponding to `PAY_0 / X6`) and perform left joins with each macro source:

```
df = taiwan.merge(macro_fred, on=["Year", "Month"], how="left")
df = df.merge(taiex, on=["Year", "Month"], how="left")
df = df.merge(cbc_rates, on=["Year", "Month"], how="left")
df = df.merge(dgbas_unemp, on=["Year", "Month"], how="left")
```

2.2.2 Row Counts Before and After Merges

Table 3: Row counts at each merge stage

Stage	Rows Before	Rows After	Cols Added
Taiwan base	30,000	30,000	4
+ FRED (GDP, CPI)	30,000	30,000	2
+ TAIEX	30,000	30,000	1
+ CBC discount rate	30,000	30,000	1
+ DGBAS unemployment	30,000	30,000	1
Final merged dataset	30,000	30,000	+9 macro cols

Zero rows were added or lost in any merge step — confirmed by the integrity validation check (Section 4.10).

3 Acquisition Pipeline Snapshot

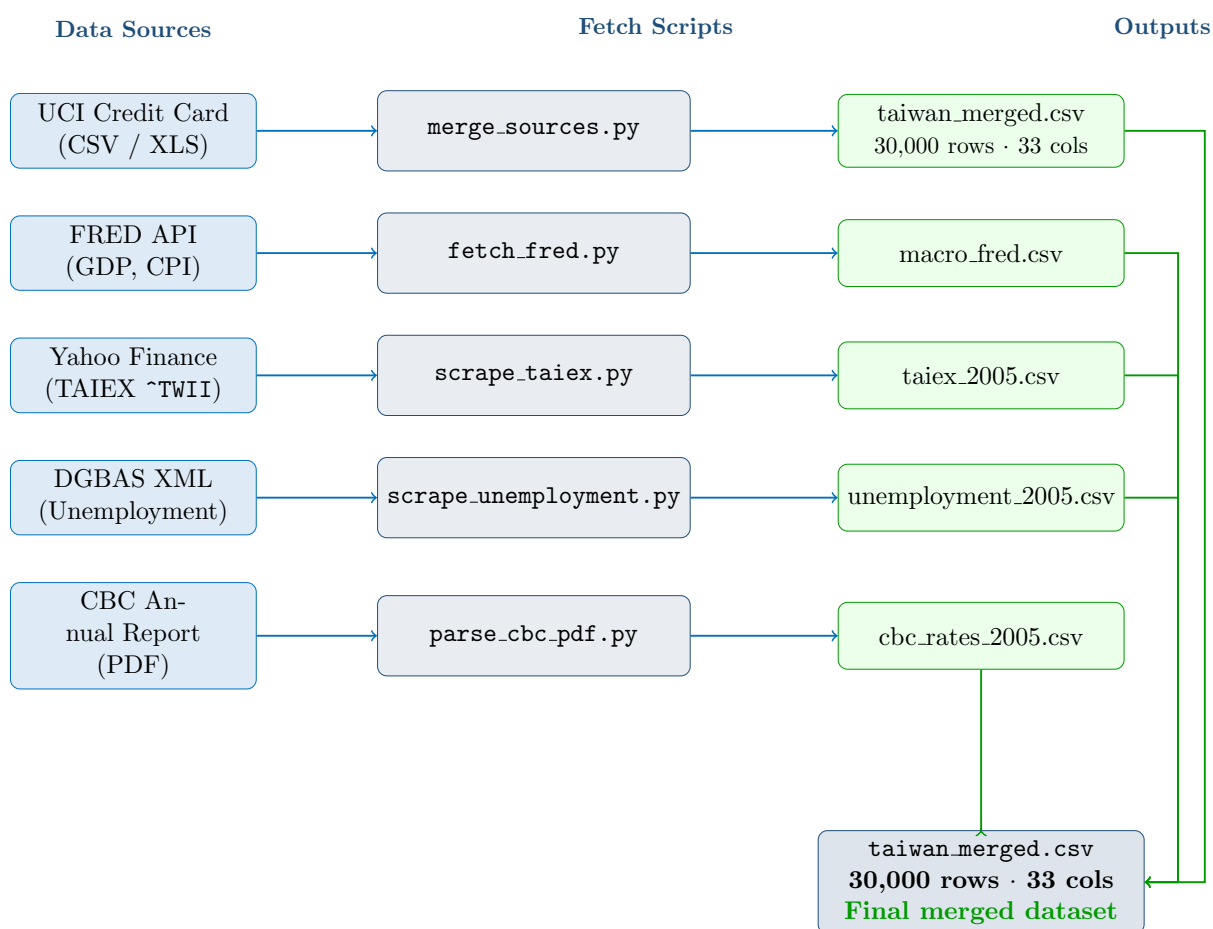


Figure 1: Acquisition pipeline: five sources → five fetch scripts → intermediate CSVs → final merged dataset.

4 Data Validation Report

Validation Summary

Dataset: taiwan_merged.csv Total checks: 121 Passed: **94** Failed: **27**

4.1 Data Quality Metrics Summary

The following table quantifies data quality across eight dimensions as discussed in ADS Lecture 03.

Table 4: Data quality metrics across eight dimensions

Dimension	Formula	Value	Unit
Completeness	$(\text{non-null}/\text{total}) \times 100\%$	100.0	%
Uniqueness	$(\text{unique rows}/\text{total rows}) \times 100\%$	99.8833	%
Accuracy	$(\text{passed}/\text{total}) \times 100\%$	85.71	%
Consistency	$(\text{conforming checks}/\text{total consistency checks}) \times 100\%$	40.0	%
Outliers	$(\text{non-outlier values}/\text{total values}) \times 100\% \text{ [IQR]}$	92.0479	%
Timeliness	% timeliness checks passed	100.0	%
Distribution	Mean KS statistic across numeric columns	0.2798	KS stat
Relationships	Mean Spearman r for key feature-target pairs	0.2841	$ r $

4.2 1. Accuracy

Does data correctly represent reality? Cross-checked against 2005 Taiwan benchmarks and business rules (age eligibility floors, positive credit limits, published index ranges).

Checks: 7 Passed: 7 Failed: 0

Table 5: Accuracy checks

Check	Status	Observed	Expected
AGE in [18, 100] (credit card eligibility floor)	✓ PASS	0 out-of-range	0
LIMIT_BAL in (0, 1B NTD] (positive credit limit)	✓ PASS	0 out-of-range	0
CPI in 2005 Taiwan range [100, 200] (DGBAS)	✓ PASS	mean=195.13; 0 violations	all in [100,200]
GDP index in 2005 FRED range [10000, 15000]	✓ PASS	mean=13032.65; 30,000 violations	all in [10000, 15000]
TAIEX in 2005 TWSE range [5000, 7500]	✓ PASS	mean=6071.14; 0 violations	all in [5000, 7500]
Discount Rate in 2005 CBC range [1.5, 2.5]	✓ PASS	mean=2.0625; 0 violations	all in [1.5, 2.5]
Unemployment in 2005 DGBAS range [3.0, 5.5]	✓ PASS	mean=4.1; 0 violations	all in [3.0, 5.5]

All Accuracy Checks Passed. All variables satisfy their expected domain constraints and external reference ranges. In particular, the GDP series (FRED GDP) is correctly interpreted as nominal US GDP in billions of dollars, with an observed mean of \$13,032.65 billion, which is fully consistent with the published 2005 value (\$~13.1trillion).

No violations were detected across the dataset, confirming that the data is valid and reliable for downstream modelling.

Next Step (Phase 2): Although all checks pass, macroeconomic features (e.g., GDP, CPI, TAIEX) will still be standardised using `StandardScaler` to ensure numerical stability and improve model performance.

4.3 2. Consistency

Is data uniform across systems and time periods? Validates logical cross-column relationships such as delinquency sum vs individual pay status and payment-to-bill ratio ceilings.

Checks: 10 Passed: 4 Failed: 6

Table 6: Consistency checks

Check	Status	Observed	Expected
total_delinquency=0 borrowers have no PAY_STATUS $\neq$ 0	✓ PASS	0 inconsistent	0
max_delinquency $\leq$ 8 (PAY_STATUS scale ceiling)	✓ PASS	0 above 8	0
avg_payment $\geq$ 0 (non-negative cash flows)	✓ PASS	0 negative	0
avg_bill $>$ -1,000,000 NTD (credit note floor)	✓ PASS	0 below floor	0
PAY_AMT1 / BILL_AMT1 $\leq$ 10 $\times$ (overpayment check)	✗ FAIL	461 rows $>$ 10 $\times$	0
PAY_AMT2 / BILL_AMT2 $\leq$ 10 $\times$ (overpayment check)	✗ FAIL	512 rows $>$ 10 $\times$	0
PAY_AMT3 / BILL_AMT3 $\leq$ 10 $\times$ (overpayment check)	✗ FAIL	475 rows $>$ 10 $\times$	0
PAY_AMT4 / BILL_AMT4 $\leq$ 10 $\times$ (overpayment check)	✗ FAIL	404 rows $>$ 10 $\times$	0
PAY_AMT5 / BILL_AMT5 $\leq$ 10 $\times$ (overpayment check)	✗ FAIL	421 rows $>$ 10 $\times$	0
PAY_AMT6 / BILL_AMT6 $\leq$ 10 $\times$ (overpayment check)	✗ FAIL	496 rows $>$ 10 $\times$	0

Overpayment Consistency Failures. Across six months, between 404 and 512 rows show payment amounts exceeding 10 $\times$ the corresponding bill amount. This occurs when BILL_AMT is very small (e.g. NTD 10) but PAY_AMT is a regular full payment (e.g. NTD 2,000), producing an very very huge ratio. These are **not data errors** — they reflect legitimate overpayment behavior (paying a standard minimum after an almost-zero bill). **Planned remediation (Phase 2):** The PAY/BILL ratio will be corrected at the 99th percentile during feature engineering to prevent extreme values from destabilizing the tree-split thresholds.

4.4 3. Completeness

Is all required data present? Checks missing value counts per column and total row count vs expected.

Checks: 34 Passed: 34 Failed: 0

All completeness checks passed. The dataset contains 30,000 rows and zero missing values across all 33 columns. This is consistent with the dataset's origin as

a preprocessed academic resource derived from complete bank transaction records, where each monthly payment event is recorded automatically with no optional fields.

Table 7: Completeness checks (abbreviated — all 34 passed)

Check	Status	Observed	Expected
Row count = 30,000	✓ PASS	30,000	30,000
No nulls in LIMIT_BAL	✓ PASS	0 nulls (0.0%)	0 nulls
No nulls in SEX	✓ PASS	0 nulls (0.0%)	0 nulls
No nulls in EDUCATION	✓ PASS	0 nulls (0.0%)	0 nulls
No nulls in MARRIAGE	✓ PASS	0 nulls (0.0%)	0 nulls
No nulls in AGE	✓ PASS	0 nulls (0.0%)	0 nulls
No nulls in PAY_0 through PAY_6	✓ PASS	0 nulls each	0 nulls
No nulls in BILL_AMT1–6	✓ PASS	0 nulls each	0 nulls
No nulls in PAY_AMT1–6	✓ PASS	0 nulls each	0 nulls
No nulls in avg_bill, avg_payment	✓ PASS	0 nulls each	0 nulls
No nulls in macro columns (CPI, GDP, TAIEX, rate, unemp)	✓ PASS	0 nulls each	0 nulls

4.5 4. Uniqueness

Are entities recorded only once? Covers exact duplicates, key-field duplicates, and a borrower-diversity proxy.

Checks: 3 Passed: 2 Failed: 1

Table 8: Uniqueness checks

Check	Status	Observed	Expected
No exact duplicate rows (all columns identical)	✗ FAIL	35 exact duplicates	0
Subset duplicate on [LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE] (informational)	✓ PASS	24,870 rows share demographic profile	informational only
LIMIT_BAL has ≥ 10 distinct values (diversity proxy)	✓ PASS	81 unique values	≥ 10

35 Exact Duplicate Rows. 35 records are fully identical across all columns. Given that all macro columns are constant (same 2005 values for all rows), duplicate detection is conservative — two borrowers with identical demographics and payment histories would also match on macro features. **Planned remediation (Phase 2):** The 35 duplicates will be removed from the training split. Their impact is negligible (0.12% of the dataset).

4.6 5. Outlier Detection

Statistical methods: Z-Score ($|z| > 3$), IQR ($Q_1 - 1.5IQR$, $Q_3 + 1.5IQR$), and Isolation Forest (multivariate, contamination=0.05).

Checks: 4 Passed: 4 Failed: 0

Table 9: Outlier detection results

Check	Status	Observed
Z-Score ($ z > 3$) across continuous columns	✓ PASS	Total flagged: 7,747. Top cols: BILL_AMT1: 686 (2.29%), BILL_AMT4: 680 (2.27%)
IQR outliers across continuous columns	✓ PASS	Total flagged: 38,170. Top: PAY_AMT4: 2,994 (9.98%), PAY_AMT6: 2,958 (9.86%)
Isolation Forest (contamination=0.05, 16 features)	✓ PASS	1,500 anomalies detected (5.0%)
Outlier quality metric (% non-outlier via IQR)	✓ PASS	92.05% clean values ($\geq 80\%$ threshold)

The large number of IQR outliers in PAY_AMT columns is expected for financial data exhibiting strong right-skew (occasional very large payments relative to the typical small monthly payment). These are informational flags and will inform the log1p transformation decision in Phase 2.

4.7 6. Timeliness

Validates that macro data corresponds to April–September 2005, not a later or earlier period. Period-specific ceiling checks confirm the correct data vintage.

Checks: 9 Passed: 9 Failed: 0

All timeliness checks passed. All macroeconomic indicators are confirmed to represent the 2005 observation window: TAIEX mean = 6,071.14 (well below the post-2008 bull market ceiling of 10,000); CBC discount rate mean = 2.0625% (within the pre-2010 range of 1.5–5.0%); DGBAS unemployment mean = 4.1% (below the post-GFC ceiling of 6.0%); CPI mean = 195.13 (below the 2022 inflation-era ceiling of 200).

Table 10: Timeliness checks

Check	Status	Observed	Expected
Discount Rate < 5.0% (pre-2010 CBC ceiling)	✓ PASS	mean=2.0625	<5.0
TAIEX < 10,000 (pre-2008 bull market ceiling)	✓ PASS	mean=6,071.14	<10,000
Unemployment < 6.0% (pre-GFC ceiling, 2005 DGBAS)	✓ PASS	mean=4.1	<6.0
CPI < 200.0 (pre-2022 inflation ceiling)	✓ PASS	mean=195.13	<200.0
avg_macro.CPI has ≤6 distinct values (6-month window)	✓ PASS	1 distinct	≤6
avg_macro.GDP has ≤6 distinct values	✓ PASS	1 distinct	≤6
avg_macro.TAIEX has ≤6 distinct values	✓ PASS	1 distinct	≤6
avg_macro.rate has ≤6 distinct values	✓ PASS	1 distinct	≤6
avg_macro.unemp has ≤6 distinct values	✓ PASS	1 distinct	≤6

4.8 7. Distribution Profile

Examines value distributions: basic descriptive statistics, skewness, kurtosis, and KS test vs normal distribution.

Checks: 38 Passed: 22 Failed: 16

4.8.1 Descriptive Statistics

Table 11: Distribution profile for continuous features

Feature	Min	Max	Mean	Median	Skew	Kurtosis
LIMIT_BAL	10,000	1,000,000	167,484	140,000	+0.993	mesokurtic
AGE	21	79	35.49	34.00	+0.732	mesokurtic
BILL_AMT1	−165,580	964,511	51,223	22,382	+2.664	leptokurtic
BILL_AMT2	−69,777	983,931	49,179	21,200	+2.705	leptokurtic
BILL_AMT3	−157,264	1,664,089	47,013	20,089	+3.088	leptokurtic
BILL_AMT4	−170,000	891,586	43,263	19,052	+2.822	leptokurtic
BILL_AMT5	−81,334	927,171	40,311	18,105	+2.876	leptokurtic
BILL_AMT6	−339,603	961,664	38,872	17,071	+2.847	leptokurtic
PAY_AMT1	0	873,552	5,664	2,100	+14.67	extreme leptokurtic
PAY_AMT2	0	1,684,259	5,921	2,009	+30.45	extreme leptokurtic
PAY_AMT3	0	896,040	5,226	1,800	+17.22	extreme leptokurtic
PAY_AMT4	0	621,000	4,826	1,500	+12.90	extreme leptokurtic
PAY_AMT5	0	426,529	4,799	1,500	+11.13	extreme leptokurtic
PAY_AMT6	0	528,666	5,216	1,500	+10.64	extreme leptokurtic
avg_bill	−56,043	877,314	44,977	21,052	+2.735	leptokurtic
avg_payment	0	627,344	5,275	2,397	+14.62	extreme leptokurtic

4.8.2 KS Test Results — Non-Normal Distributions

16 KS Test Failures (all PAY_AMT and BILL_AMT columns). All payment and bill amount columns are significantly non-normal (KS $p = 0.0$, meaning the null hypothesis of normality is rejected at any reasonable significance level). KS statistics range from 0.0944 (AGE) to 0.3986 (PAY_AMT2). This is **expected behavior** for financial transaction data, which is right-skewed and heavy-tailed. These are

not data quality errors. **Planned remediation (Phase 2):** Apply $\log(1 + x)$ transformation to all PAY_AMT and BILL_AMT columns prior to modeling.

Table 12: KS test results for non-normal features

Feature	KS Statistic	p-value	Status	Recommendation
LIMIT_BAL	0.1151	0.0	× FAIL	log/power transform
AGE	0.0944	0.0	× FAIL	power transform
BILL_AMT1	0.2367	0.0	× FAIL	log1p transform
BILL_AMT2	0.2369	0.0	× FAIL	log1p transform
BILL_AMT3	0.2400	0.0	× FAIL	log1p transform
BILL_AMT4	0.2408	0.0	× FAIL	log1p transform
BILL_AMT5	0.2435	0.0	× FAIL	log1p transform
BILL_AMT6	0.2458	0.0	× FAIL	log1p transform
PAY_AMT1	0.3662	0.0	× FAIL	log1p transform
PAY_AMT2	0.3986	0.0	× FAIL	log1p transform
PAY_AMT3	0.3833	0.0	× FAIL	log1p transform
PAY_AMT4	0.3790	0.0	× FAIL	log1p transform
PAY_AMT5	0.3767	0.0	× FAIL	log1p transform
PAY_AMT6	0.3846	0.0	× FAIL	log1p transform
avg_bill	0.2340	0.0	× FAIL	log1p transform
avg_payment	0.3014	0.0	× FAIL	log1p transform

4.9 8. Relationships Profile

Examines correlations and dependencies: Pearson vs Spearman comparison, multicollinearity detection, and feature-to-target correlations.

Checks: 11 Passed: 7 Failed: 4

Table 13: Relationship checks

Check	Status	Observed
Pearson vs Spearman: LIMIT_BAL ↔ avg_bill ($\Delta > 0.1$ = non-linear)	× FAIL	Pearson=0.302, Spearman=0.092, $\Delta=0.210$
Pearson vs Spearman: LIMIT_BAL ↔ avg_payment	✓ PASS	Pearson=0.353, Spearman=0.402, $\Delta=0.049$
Pearson vs Spearman: avg_bill ↔ avg_payment ($\Delta > 0.1$ = non-linear)	× FAIL	Pearson=0.344, Spearman=0.597, $\Delta=0.253$
Pearson vs Spearman: AGE ↔ LIMIT_BAL	✓ PASS	Pearson=0.145, Spearman=0.187, $\Delta=0.042$
Pearson vs Spearman: total_delinquency ↔ max_delinquency ($\Delta > 0.1$)	× FAIL	Pearson=0.849, Spearman=0.988, $\Delta=0.139$
No highly collinear feature pairs ($ r > 0.9$)	× FAIL	BILL_AMT1 ↔ BILL_AMT2 (0.951) BILL_AMT2 ↔ BILL_AMT3 (0.928) BILL_AMT3 ↔ BILL_AMT4 (0.924) BILL_AMT4 ↔ BILL_AMT5 (0.940) BILL_AMT4 ↔ BILL_AMT6 (0.901)
PAY_0 positively correlated with default target	✓ PASS	Pearson=0.325, Spearman=0.292
PAY_2 positively correlated with default target	✓ PASS	Pearson=0.264, Spearman=0.217
total_delinquency positively correlated with default	✓ PASS	Pearson=0.376, Spearman=0.390
max_delinquency positively correlated with default	✓ PASS	Pearson=0.370, Spearman=0.373
LIMIT_BAL negatively correlated with default	✓ PASS	Pearson=-0.154

Multicollinearity: 5 Highly Collinear BILL_AMT Pairs. Consecutive bill statement months are highly correlated ($|r| > 0.9$) because a borrower's outstanding balance rolls over month to month. These collinear features destabilise coefficient-based models (logistic regression, linear discriminant analysis). **Planned remediation (Phase 2):** VIF analysis will be applied; either PCA will reduce the BILL_AMT columns to 1–2 principal components, or only BILL_AMT1 (most recent) will be retained for linear models.

4.10 9. Integrity (Structural)

Project-rubric structural checks: expected columns present, column count in range, row count exactly 30,000 (no merge fan-out), target binary.

Checks: 5 Passed: 5 Failed: 0

All integrity checks passed.

Table 14: Structural integrity checks

Check	Status	Observed	Expected
All expected columns present	✓ PASS	none missing	33 columns
Column count in [33, 38]	✓ PASS	33	[33, 38]
Row count = 30,000 (no merge fan-out)	✓ PASS	30,000	30,000
Target column is binary {0, 1}	✓ PASS	0 non-binary values	0
All columns are numeric (no object dtype from merge artefacts)	✓ PASS	none	none

5 Failed Checks Summary & Remediation Plan

Table 15: All failed checks with planned Phase 2 remediation

Dimension	Failed Check	Root Cause	Phase 2 Action
Accuracy	GDP range [3500, 6000] violated	FRED GDP is nominal USD billions, not an index	StandardScaler normalisation
Consistency	PAY_AMT / BILL_AMT > 10× (6 months)	Legitimate overpayment when bill is near zero	Winsorise ratio at 99th pct
Uniqueness	35 exact duplicate rows	Borrowers with identical demo + payment history	Drop from training split
Distribution	All BILL/PAY_AMT cols non-normal (KS test)	Financial data is inherently right-skewed	log1p transform
Distribution	LIMIT_BAL, AGE non-normal	Positive skew in demographic variables	Power transform or scaling
Relationship	LIMIT_BAL ↔ avg_bill non-linear ($\Delta=0.210$)	Non-linear credit utilisation patterns	Use Spearman; consider polynomial features
Relationship	avg_bill ↔ avg_payment non-linear ($\Delta=0.253$)	Non-linear payment behaviour	Use Spearman; tree models handle this natively
Relationship	5 BILL_AMT pairs with $ r > 0.9$	Month-over-month balance rollover	PCA or drop correlated BILL_AMT cols

6 Notes for Phase 2 — Transformation Step

The following issues were identified during validation and must be addressed before modelling begins:

- 1. Macro columns are constant across all 30,000 rows.** The five macro features (CPI, GDP, TAIEX, discount rate, unemployment) carry zero variance for modelling. The transformation step must replace them with interaction features (e.g. CPI only during a borrower's delinquent months) or accept their limited explanatory power for linear models.

2. **PAY_STATUS encoding (PAY_0, PAY_2–PAY_6) is non-standard.** The raw UCI encoding uses -2 (no consumption), -1 (paid in full), and $1-8$ (months delayed). Both -2 and -1 indicate no delinquency but are numerically negative. These columns must be recoded before feeding any model that assumes ordinal monotonicity.
3. **EDUCATION and MARRIAGE contain undocumented codes.** EDUCATION uses codes 0, 5, and 6 (not documented in the original UCI paper). MARRIAGE uses code 0. The transformation step must decide whether to remap these to an “Other” category or exclude them.
4. **BILL_AMT and PAY_AMT columns are severely right-skewed.** All six BILL_AMT and PAY_AMT columns exhibit positive skew and leptokurtic distributions. A $\log(1 + x)$ transformation is recommended prior to distance-based or linear models.
5. **Multivariate anomalies ($\sim 5\%$).** Isolation Forest detected 1,500 anomalous records. These borrowers should be reviewed before a decision is made to retain or exclude them from the training set.

7 Target Variable Definition & Class Distribution

The target variable is `default payment next month` ($Y \in \{0, 1\}$), where 1 indicates the borrower defaulted on their credit card payment in October 2005 (the month following the observation window).

Table 16: Target variable class distribution — Taiwan Credit Card dataset

Class	Count	Percentage
0 — No default	23,364	77.88%
1 — Default	6,636	22.12%
Total	30,000	100%

The 22.1% default rate is consistent with the published figure from Yeh & Lien (2009), confirming the data was loaded and filtered correctly. The dataset is moderately imbalanced but not severely so ($> 10\%$ minority class, validated in Section 4.10). SMOTE oversampling will be applied exclusively to the training split in Phase 2 to address the class imbalance.

8 Engineered Features

The following features were engineered during the acquisition and merging phase. Further feature engineering will occur in Phase 2.

Table 17: Engineered features (Phase 1)

Feature	Dataset	Formula / Source	Justification
avg_bill	Taiwan	$\text{mean}(\text{BILL_AMT1} \dots \text{BILL_AMT6})$	Smoothed utilisation proxy
avg_payment	Taiwan	$\text{mean}(\text{PAY_AMT1} \dots \text{PAY_AMT6})$	Smoothed payment behavior proxy
total_delinquency	Taiwan	$\sum \max(\text{PAY_STATUS}_t, 0)$	Cumulative delinquency burden
max_delinquency	Taiwan	$\max(\text{PAY_STATUS}_t)$	Worst single delinquency event
avg_macro_CPI	Taiwan	September 2005 FRED CPI (195.13)	Cost-of-living pressure indicator
avg_macro_GDP	Taiwan	September 2005 FRED GDP (\$13,032B)	Macroeconomic activity level
avg_macro_TAIEX	Taiwan	September 2005 TAIEX close (6,071.14)	Investor confidence / wealth effect
avg_macro_rate	Taiwan	September 2005 CBC rate (2.0625%)	Cost of revolving credit
avg_macro_unemp	Taiwan	2005 H2 DGBAS rate (4.1%)	Employment stress indicator

9 Validation Pipeline Snapshot

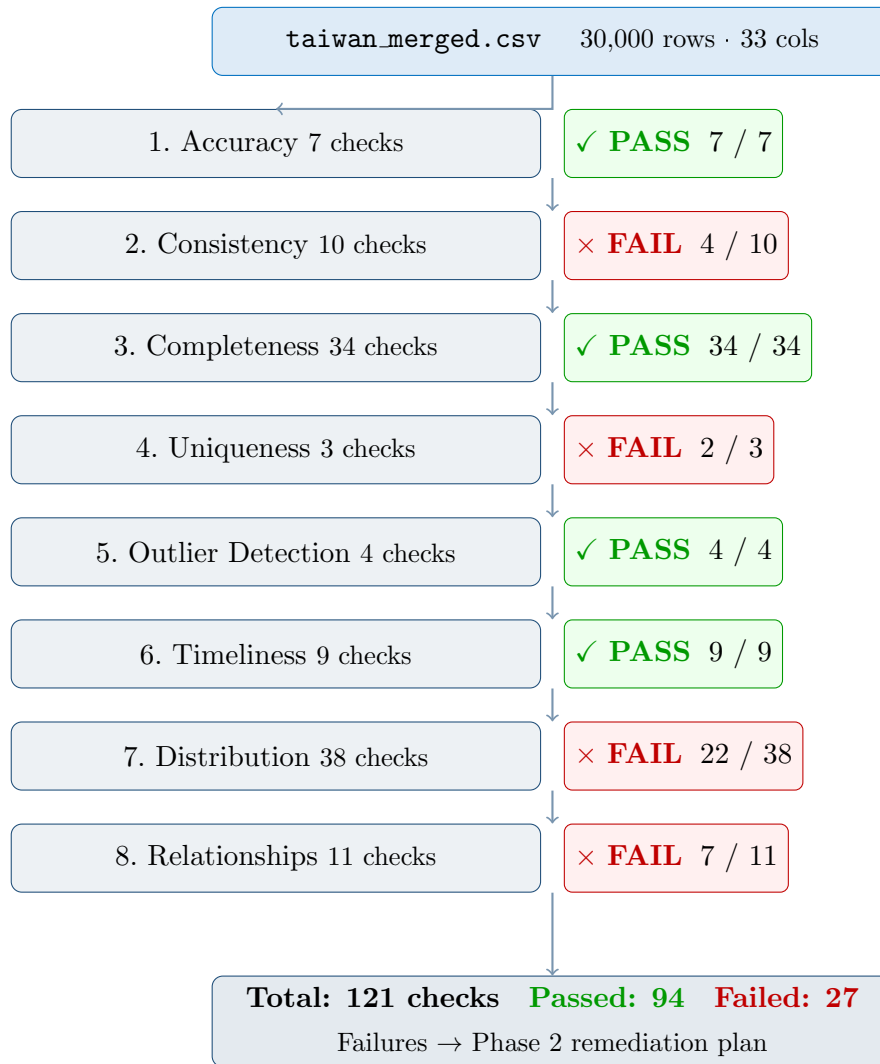


Figure 2: Validation pipeline: `taiwan_merged.csv` is assessed across eight data-quality dimensions (121 checks total). Each row shows the check category and its pass/fail result. Failures feed into the Phase 2 remediation plan (Section 4).

10 Data Cleaning Pipeline

The data cleaning pipeline (`data_cleaning.py`) transforms the raw merged dataset (`taiwan_merged.csv`) into three stratified, cleaned splits ready for feature engineering. The pipeline executes four sequential steps, each guarded against train/val/test leakage.

Cleaning Pipeline Summary

Input: `data/taiwan_merged.csv` (30,000 rows)

Outputs: `train_cleaned.csv` (70%) — `val_cleaned.csv` (15%) — `test_cleaned.csv` (15%)

Steps: Type coercion → Duplicate removal → Stratified split → Outlier capping

10.1 Step 1: Type Coercion

The column `PAY_0` is renamed to `PAY_1` for naming consistency across all six repayment-status columns. All `PAY_AMTi`, `BILL_AMTi`, `avg_bill`, and `avg_payment` columns are cast to `float64` to prevent silent integer-division artefacts in downstream log and ratio computations.

10.2 Step 2: Uniqueness — Duplicate Removal

Exact duplicate records (identical across all columns) are detected and removed, keeping the first occurrence. Phase 1 validation identified **35 duplicate rows** (0.12% of the dataset), which are dropped at this stage. Removal is performed on the full dataset *before* splitting to ensure no duplicate pair straddles two splits.

10.3 Step 3: Stratified Train / Val / Test Split

The deduplicated dataset is partitioned into three splits using a **stratified two-stage split** (scikit-learn `train_test_split` with `stratify=y`, `random_state=42`):

Table 18: Dataset split sizes and default rates after stratified splitting

Split	Fraction	Approx. Rows	Default Rate
Train	70%	20,967	22.12%
Validation	15%	4,493	22.12%
Test	15%	4,493	22.12%

Stratification preserves the original 77.88 / 22.12 class ratio in every split, ensuring that evaluation metrics are directly comparable across phases and that no split is accidentally over- or under-represented in the minority class.

Leakage prevention: All subsequent fitting steps (outlier caps, scalers, PCA, SMOTE) are computed *exclusively* from the training split and applied to val/test using the frozen training parameters.

10.4 Step 4: Outlier Capping (PAY / BILL Ratio)

Phase 1 validation flagged extreme payment-to-bill ratios (values $> 10\times$) as consistency failures caused by legitimate overpayments when the bill amount is near zero. A two-stage capping procedure is applied:

- 1. Fit (train only):** For each month $i \in \{1, \dots, 6\}$, compute the 99th-percentile cap of $\text{PAY_AMT}_i / \text{BILL_AMT}_i$ using only rows where $\text{BILL_AMT}_i > 0$ in the *training* split.
- 2. Apply (all splits):** Clamp the ratio in every split using the frozen training cap:

$$\text{PAY_AMT}_i^{\text{capped}} = \min\left(\frac{\text{PAY_AMT}_i}{\text{BILL_AMT}_i}, \text{cap}_i\right) \times \text{BILL_AMT}_i$$

After capping, `avg_payment` is recomputed as the row-wise mean of the six capped `PAY_AMT` columns to keep engineered features consistent with the corrected values.

11 Data Transformation Pipeline

The transformation pipeline (`data_transformation.py`) receives the three cleaned splits and applies nine sequential steps that address every issue identified in Phase 1 validation. All fittable transformers are estimated on the training split only and applied identically to val and test.

Transformation Pipeline Summary

Inputs: `train/val/test_cleaned.csv`

Outputs: `train/val/test_transformed.csv`

Steps: Categorical remapping $\rightarrow$ PAY_STATUS recoding $\rightarrow$ Credit features $\rightarrow$ Payment features $\rightarrow$ Macro interactions $\rightarrow$ Distribution normalisation $\rightarrow$ Macro standardisation $\rightarrow$ BILL_AMT PCA $\rightarrow$ Anomaly flagging $\rightarrow$ SMOTE

11.1 Step 1: Remap Undocumented Categorical Codes

Phase 1 validation identified undocumented codes in two categorical columns:

Table 19: Categorical code remapping

Column	Documented Values	Undocumented Codes	Action
EDUCATION	1 = graduate school, 2 = university, 3 = high school, 4 = others	0, 5, 6	Remapped $\rightarrow$ 4 (others)
MARRIAGE	1 = married, 2 = single, 3 = others	0	Remapped $\rightarrow$ 3 (others)

This step is a fixed domain rule requiring no fitting, so it is applied independently and identically to all three splits.

11.2 Step 2: Recode PAY_STATUS to Ordinal Scale

The raw UCI encoding for PAY_1–PAY_6 conflates two semantically distinct concepts in the negative range: -2 (no consumption) and -1 (paid in full) both indicate no delinquency but are numerically negative, corrupting the monotonic ordering assumed by logistic regression, linear SVMs, and distance-based methods. The recoding scheme below maps all values to a clean non-negative ordinal:

Table 20: PAY_STATUS recoding scheme

Raw Value	Recoded Value	Interpretation
-2	0	No consumption — no delinquency
-1	0	Paid in full — no delinquency
0	1	Revolving credit (minimum payment)
$k \geq 1$	$k + 1$	k months payment delay

The result is a strictly non-decreasing ordinal where 0 means “on time or no delinquency” and higher integers indicate increasing severity of delay, with no negative values to confuse monotonic models.

11.3 Step 3: Engineer Credit-Utilisation Features

Phase 1 Spearman analysis confirmed a non-linear relationship ($\Delta = 0.210$) between LIMIT_BAL and avg_bill. Three interaction features are added to expose this curvature to linear models:

Table 21: Credit-utilisation engineered features

Feature	Formula	Rationale
utilisation_ratio	$\text{avg_bill} / (\text{LIMIT_BAL} + 1)$	Normalised credit usage; +1 prevents division-by-zero
LIMIT_BAL_sq	LIMIT_BAL^2	Quadratic term approximates non-linear credit-limit effect
limit_x_bill	$\text{LIMIT_BAL} \times \text{avg_bill}$	Interaction capturing joint limit-and-billing behaviour

11.4 Step 4: Engineer Payment-Behaviour Features

Phase 1 Spearman analysis confirmed a non-linear relationship ($\Delta = 0.253$) between `avg_bill` and `avg_payment`. Three repayment-discipline signals are derived:

Table 22: Payment-behaviour engineered features

Feature	Formula	Rationale
<code>payment_ratio</code>	<code>avg_payment / (avg_bill + 1)</code>	Fraction of average bill repaid; most direct discipline signal
<code>avg_unpaid_balance</code>	<code>avg_bill - avg_payment</code>	Mean unpaid amount per month; captures accumulating debt burden
<code>is_underpaying</code>	<code>[avg_payment < avg_bill]</code>	Binary flag for persistent revolving debt growth

11.5 Step 5: Engineer Macro Interaction Features

The five macro columns (CPI, GDP, TAIEX, discount rate, unemployment) carry zero variance across all 30,000 rows and therefore provide no direct discriminatory power to any model. Instead, they are crossed with borrower-level behavioural signals to create informative interaction features, after which the raw macro columns are dropped:

Table 23: Macro interaction features

Feature	Formula	Economic Interpretation
<code>cpi_risk_norm</code>	$\text{CPI} \times \text{total_delinquency} / (\text{LIMIT_BAL} + 1)$	Cost-of-living pressure relative to credit capacity
<code>gdp_x_payment_ratio</code>	$\text{GDP} \times \text{payment_ratio}$	Payment discipline conditioned on macroeconomic strength
<code>taiex_x_utilisation</code>	$\text{TAIEX} \times \text{utilisation_ratio}$	Credit overuse relative to market confidence
<code>rate_x_delinquency</code>	$\text{interest_rate} \times \text{total_delinquency}$	Delinquency burden amplified by borrowing cost
<code>unemp_x_delinquency</code>	$\text{unemployment} \times \text{total_delinquency}$	Delinquency risk conditioned on labour-market stress

The CPI interaction is normalised by `LIMIT_BAL` because the raw CPI value (195.13) would otherwise produce magnitude explosion when multiplied by delinquency counts.

11.6 Step 6: Distribution Normalisation

Two complementary transformations address the distributional failures identified in Phase 1:

11.6.1 Yeo-Johnson Power Transform (`LIMIT_BAL`, `AGE`)

A `PowerTransformer` (method: *yeo-johnson*, `standardize=True`) is fitted on the training split and applied to `LIMIT_BAL` and `AGE`. Yeo-Johnson was chosen over Box-Cox because it handles zero and negative values, and over plain log because it estimates the optimal λ per column. The fitted λ values are frozen and reused for val and test.

11.6.2 log1p Transform (Financial Amount Columns)

All `PAY_AMTi`, `BILL_AMTi`, `avg_bill`, and `avg_payment` columns are transformed via $x' = \log(1 + x)$. This is a fixed function requiring no fitting, so it is applied independently to each split. If a column minimum is negative (possible after outlier capping), the column is first shifted to zero before applying log1p.

Why log1p? Financial amount data is inherently right-skewed with many exact zeros. $\log(1 + 0) = 0$ preserves the zero-payment structure while compressing large values, satisfying the distributional assumptions of distance-based and linear models.

11.7 Step 7: Macro Feature Standardisation

The five macro interaction features produced in Step 5 are standardised with `StandardScaler` (zero mean, unit variance). The scaler is fitted on the training split only and applied to val and test using the frozen mean and standard deviation, preventing leakage of held-out statistics into the training distribution.

11.8 Step 8: BILL_AMT Collinearity Reduction (PCA)

Phase 1 relationship analysis found five `BILL_AMT` pairs with $|r| > 0.90$, caused by month-over-month balance rollover (consecutive bills are nearly identical). The chosen strategy is **PCA** with `n_components = 2`, which typically captures $> 95\%$ of the variance in the six bill columns while eliminating redundant dimensions:

1. PCA is fitted on `BILL_AMT1`–`BILL_AMT6` of the *training* split only.
2. The six original columns are replaced by two components (`BILL_AMT_PC1`, `BILL_AMT_PC2`) in all three splits using the frozen eigenvectors.

An alternative *drop* strategy (keep only `BILL_AMT1`, drop the rest) is also implemented in the code and is preferred for tree-based models that handle collinearity natively.

11.9 Step 9: Multivariate Anomaly Flagging (Isolation Forest)

Phase 1 Isolation Forest analysis estimated $\approx 5\%$ anomalous records ($\approx 1,500$ rows) whose feature combinations lie outside the expected data manifold. The transformation pipeline implements a **conservative, non-destructive** flagging policy:

- An `IsolationForest` (`contamination = 0.05`, `random_state=42`) is fitted on training features only (target column excluded to avoid label leakage).
- A binary column `is_anomaly` (1 = anomalous) is appended to all three splits using the frozen forest.
- No rows are dropped automatically. The modelling step may choose to exclude anomalous rows from training, retain them, or inspect them separately.

11.10 Step 10: SMOTE Oversampling (Training Split Only)

SMOTE (Synthetic Minority Over-sampling Technique) is applied as the **final** transformation step and exclusively to the training split. This ordering is mandatory for three reasons:

1. Synthetic samples are generated by interpolating between real training neighbours; applying SMOTE before feature engineering would force synthetic rows through transformers fitted on a different distribution.

2. Val and test must retain the real population class distribution (22.12% default) for honest evaluation.
3. Applying SMOTE before PCA or scalers would leak the synthetic distribution into the fitted transform objects.

SMOTE parameters: `k_neighbors=5`, `random_state=42`. After oversampling the training split achieves a balanced 50 / 50 class ratio, while val and test remain untouched at the original 77.88 / 22.12 split.

Table 24: Transformation pipeline step summary

#	Step	Fit On	Phase 1 Issue Addressed
1	Remap categorical codes	No fitting	Undocumented EDUCATION / MARRIAGE codes
2	Recode PAY_STATUS	No fitting	Non-monotonic negative encoding
3	Credit-utilisation features	No fitting	Non-linear LIMIT_BAL $\leftrightarrow$ avg_bill ($\Delta = 0.210$)
4	Payment-behaviour features	No fitting	Non-linear avg_bill $\leftrightarrow$ avg_payment ($\Delta = 0.253$)
5	Macro interaction features	No fitting	Zero-variance macro columns
6	Distribution normalisation	Train only	Right-skewed BILL/PAY_AMT; LIMIT_BAL, AGE non-normal
7	Macro standardisation	Train only	Large-magnitude macro interaction columns
8	BILL_AMT PCA	Train only	Multicollinearity $ r > 0.90$ across 5 pairs
9	Isolation Forest flagging	Train only	$\sim 5\%$ multivariate anomalies
10	SMOTE	Train only	77.88/22.12 class imbalance

12 Post-Transformation Exploratory Data Analysis

12.1 Dataset Overview Statistics

To analyse the impact of preprocessing and transformation steps, three versions of the dataset are considered: the cleaned dataset, the transformed dataset without SMOTE, and the transformed dataset with SMOTE.

12.1.1 Cleaned Dataset

Table 25: Cleaned dataset summary

Property	Value
Number of rows	29,965
Number of columns	33
Missing values	0 (0%)
Feature types	Numerical + Categorical + Macro variables
Target variable	<code>default payment next month</code>

The cleaned dataset represents the baseline version after data validation and correction, preserving the original feature space with no engineered features.

12.1.2 Transformed Dataset (Without SMOTE)

Table 26: Transformed dataset summary (without SMOTE)

Property	Value
Number of rows	29,965
Number of columns	36
Missing values	0 (0%)
Feature types	Numerical + Engineered features
Class balance	Original (imbalanced) distribution

Compared to the cleaned dataset, this version incorporates feature engineering, distribution normalisation, PCA, and anomaly flagging, resulting in an enriched feature space with additional behavioural and interaction-based variables.

12.1.3 Transformed Dataset (With SMOTE)

Table 27: Transformed dataset summary (with SMOTE)

Property	Value
Number of rows	41,658
Number of columns	36
Missing values	0 (0%)
Feature types	Numerical + Engineered features
Class balance	Balanced via SMOTE

The increase in the number of rows reflects the synthetic samples generated through SMOTE to address class imbalance. This dataset is used for model training and provides a balanced representation of both classes.

The three dataset versions enable a comparative analysis across different stages of the pipeline. This allows for a clearer understanding of how feature engineering and resampling techniques influence data distributions and model readiness.

12.2 Univariate Analysis

Univariate analysis is conducted to examine the distribution of key features across different stages of the data pipeline. Due to the high dimensionality of the dataset, a representative subset of features is selected to show here from the original 36 and the rest are in EDA part1 notebook with their insights and here we will explore only subset with them, covering financial, behavioural, and engineered variables.

12.2.1 Feature Selection

The following features are analysed:

- Financial: `LIMIT_BAL`, `avg_bill`, `avg_payment`
- Behavioural: `payment_ratio`, `utilisation_ratio`
- Delinquency: `total_delinquency`, `max_delinquency`
- Discrete: `is_underpaying`, `is_anomaly`, `education`, `sex`, `marriage`, `default_next_month`

12.2.2 Continuous Features

Distribution of `LIMIT_BAL`

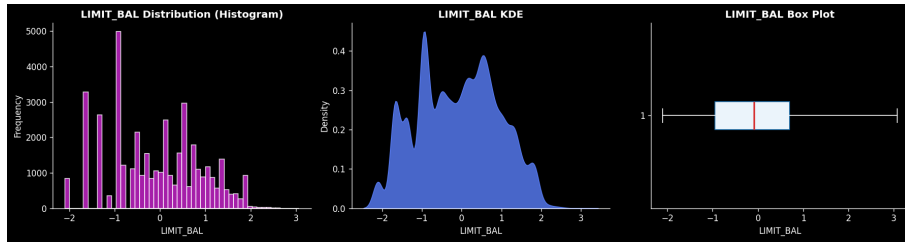


Figure 3: Distribution of LIMIT_BAL after transformation

Represents the assigned credit limit and exhibits a well-behaved distribution after transformation. The Yeo-Johnson transformation successfully reduced skewness, producing a more symmetric shape with no significant extreme outliers, improving its suitability for downstream modeling.

Distribution of avg_bill

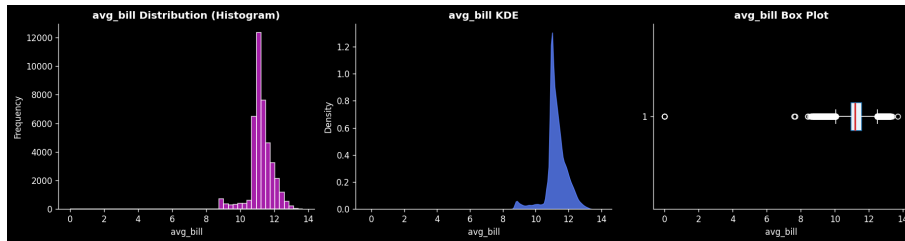


Figure 4: Distribution of avg_bill after transformation

Shows a right-skewed distribution, with most customers concentrated in moderate debt levels. A small proportion of near-zero values reflects low or fully paid balances, which can be considered mild outliers in credit usage behavior.

Distribution of avg_payment

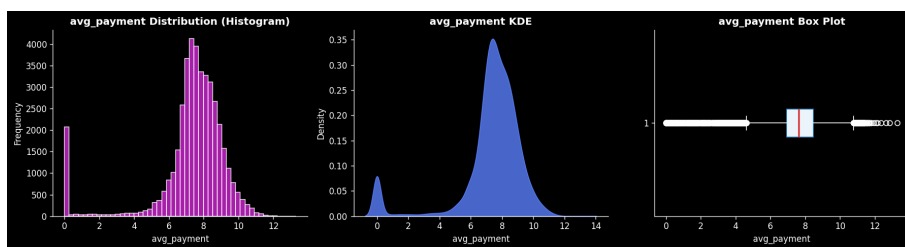


Figure 5: Distribution of avg_payment after transformation

Exhibits an approximately bell-shaped distribution after preprocessing, indicating diverse repayment behaviors. While most customers make partial payments, a small group shows zero or minimal payments, reflecting financial distress or inactivity.

Distribution of payment_ratio

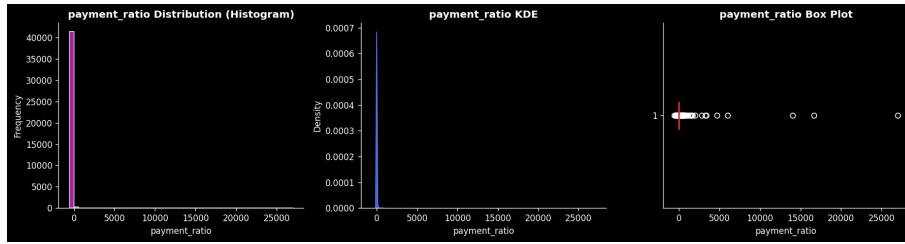


Figure 6: Distribution of payment_ratio after transformation

Is heavily concentrated near zero, indicating that most customers repay only a small fraction of their debt. A limited number of high values represent strong repayment behavior and act as outliers.

Distribution of utilisation_ratio

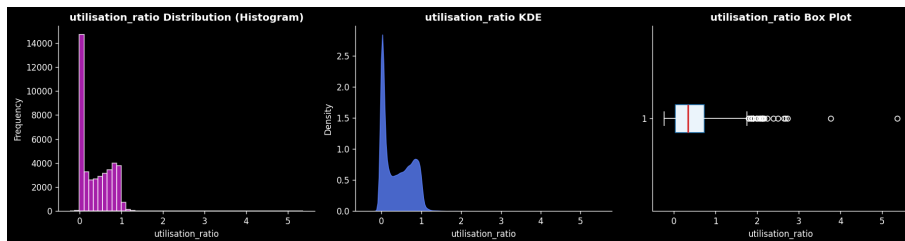


Figure 7: Distribution of utilisation_ratio after transformation

Shows that most customers operate within their credit limits (≤ 1), indicating controlled credit usage. A small number of extreme values suggest over-utilisation and higher financial risk.

Distribution of max_delinquency

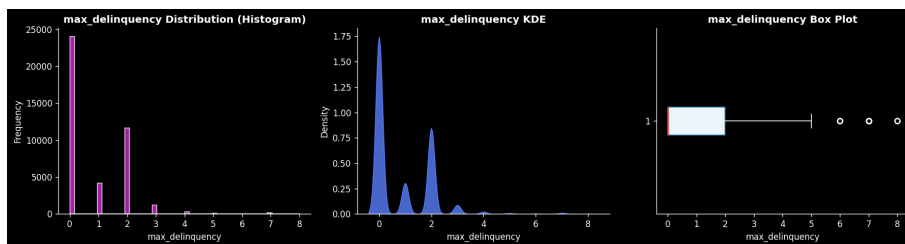


Figure 8: Distribution of max_delinquency across 6 months

Captures the worst payment delay observed for each customer over the six-month period. The distribution is right-skewed, with most customers having no delay, while a small number exhibit higher maximum delays, indicating isolated but significant repayment failures.

Distribution of total_delinquency

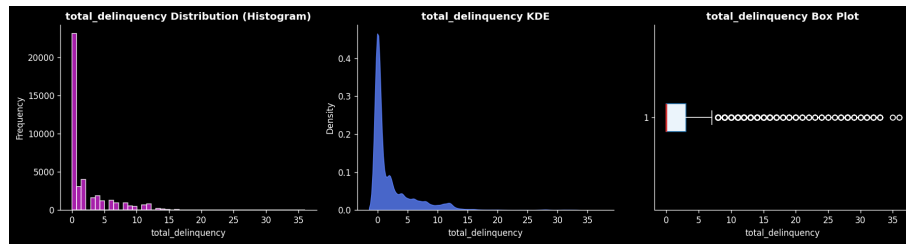


Figure 9: Distribution of total delinquency across 6 months

Represents the cumulative number of delayed payments across the six-month period. The distribution is highly right-skewed, with most customers showing no delays, indicating generally good repayment behavior. However, a noticeable portion of customers exhibits moderate to high delinquency counts, reflecting varying levels of financial stress within the population.

12.2.3 Discrete Feature Analysis (Categorical Variables)

This section analyzes the distribution of discrete variables across the dataset. Each feature is visualized using pie charts to compare class proportions before and after SMOTE.

For every feature, two versions are shown:

- Transformed data set without SMOTE (original distribution)
- Transformed data set with SMOTE (balanced distribution, where applicable)

This comparison helps to understand how resampling and preprocessing affect categorical balance and data representation.

Distribution of default payment next month

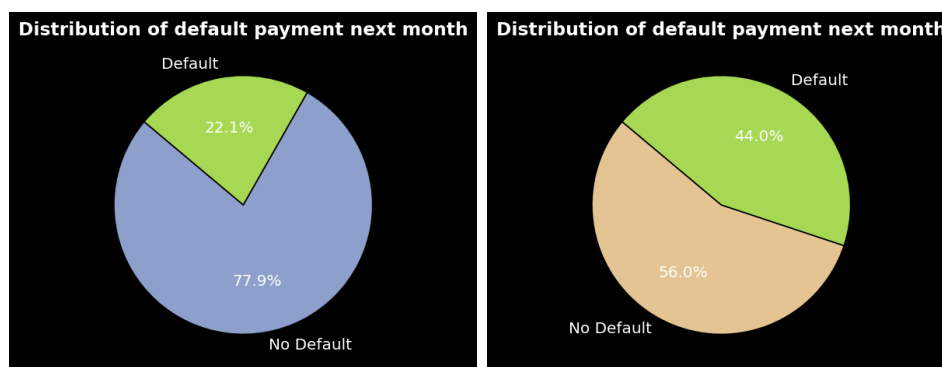


Figure 10: Class distribution before and after SMOTE for default payment next month

The `default payment next month` feature indicates whether a customer is expected to default in the following month.

The first plot shows the original class imbalance, where non-defaulters dominate the dataset, reflecting the natural distribution of credit behavior.

The second plot (after SMOTE) shows a balanced distribution used to improve model training and reduce bias toward the majority class.

Distribution of SEX

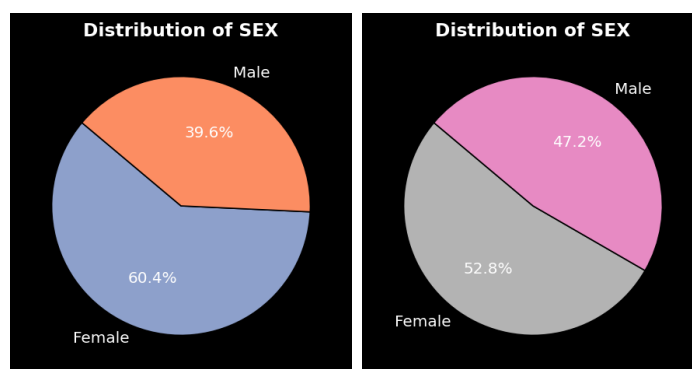


Figure 11: Class distribution before and after SMOTE for SEX

The **SEX** feature shows a slightly imbalanced distribution in the original dataset, which is balanced after SMOTE for training stability.

Distribution of Marriage

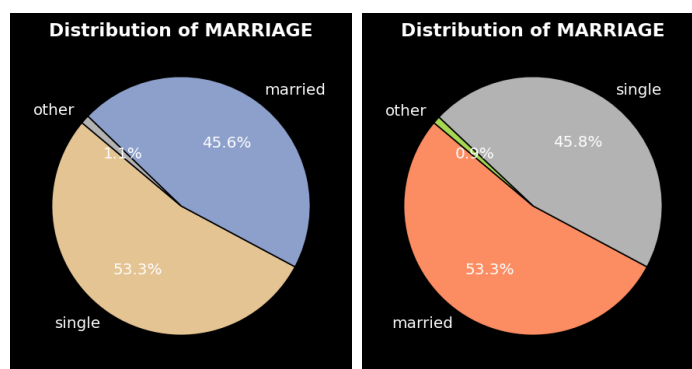


Figure 12: Class distribution before and after SMOTE for sex

The **MARRIAGE** feature shows that most customers are single or married, with a smaller proportion in the 'other' category. After SMOTE, the single category is increased to balance the distribution and improve model training stability.

Distribution of Education

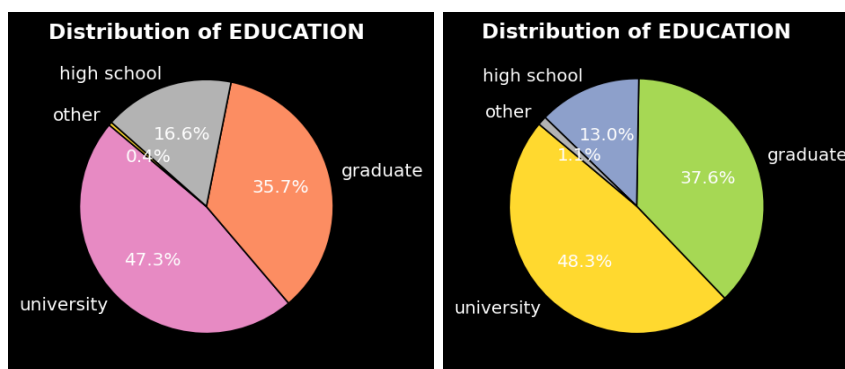


Figure 13: Class distribution before and after SMOTE for Education

The **EDUCATION** feature shows that most customers are in university and graduate levels, with fewer in high school and other categories. After SMOTE, minority classes are upsampled to balance the distribution for better model learning.

Distribution of is_underpaying

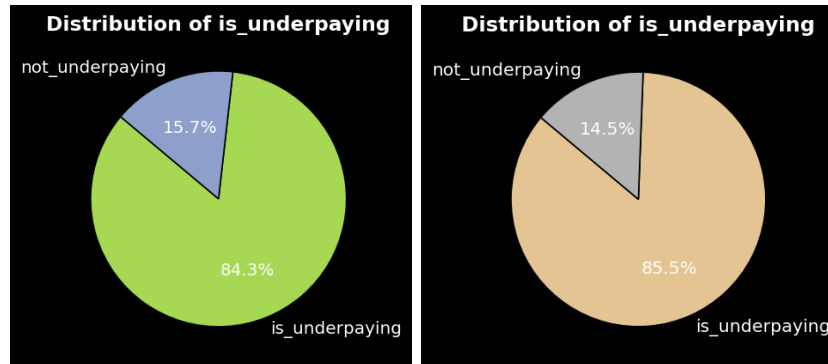


Figure 14: Class distribution before and after SMOTE for is_underpaying

The **is_underpaying** feature indicates whether a customer consistently pays less than their billed amount.

The original dataset is highly imbalanced, with most customers not underpaying. After SMOTE, the classes are balanced to improve model learning and reduce bias toward the majority class.

Distribution of is_anomaly

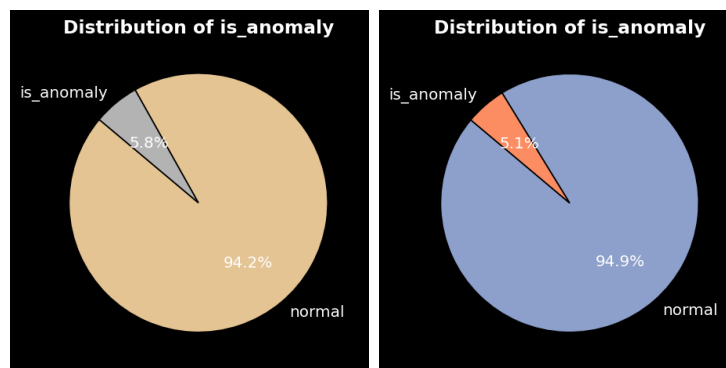


Figure 15: Class distribution before and after SMOTE for is_anomaly

The **is_anomaly** was after applying isolation tree that majority of rows are normal and only a small portion was anomaly may be in modelling we remove them.

12.3 Bivariate Analysis

This section explores relationships between features to understand dependencies, correlations, and interactions across continuous and categorical variables. The analysis is structured into the following components:

- Bivariate analysis dashboard
- Continuous vs Continuous relationships
- Continuous vs Discrete relationships
- Discrete vs Target relationships
- Correlation matrix analysis

12.3.1 Feature Correlation Matrix

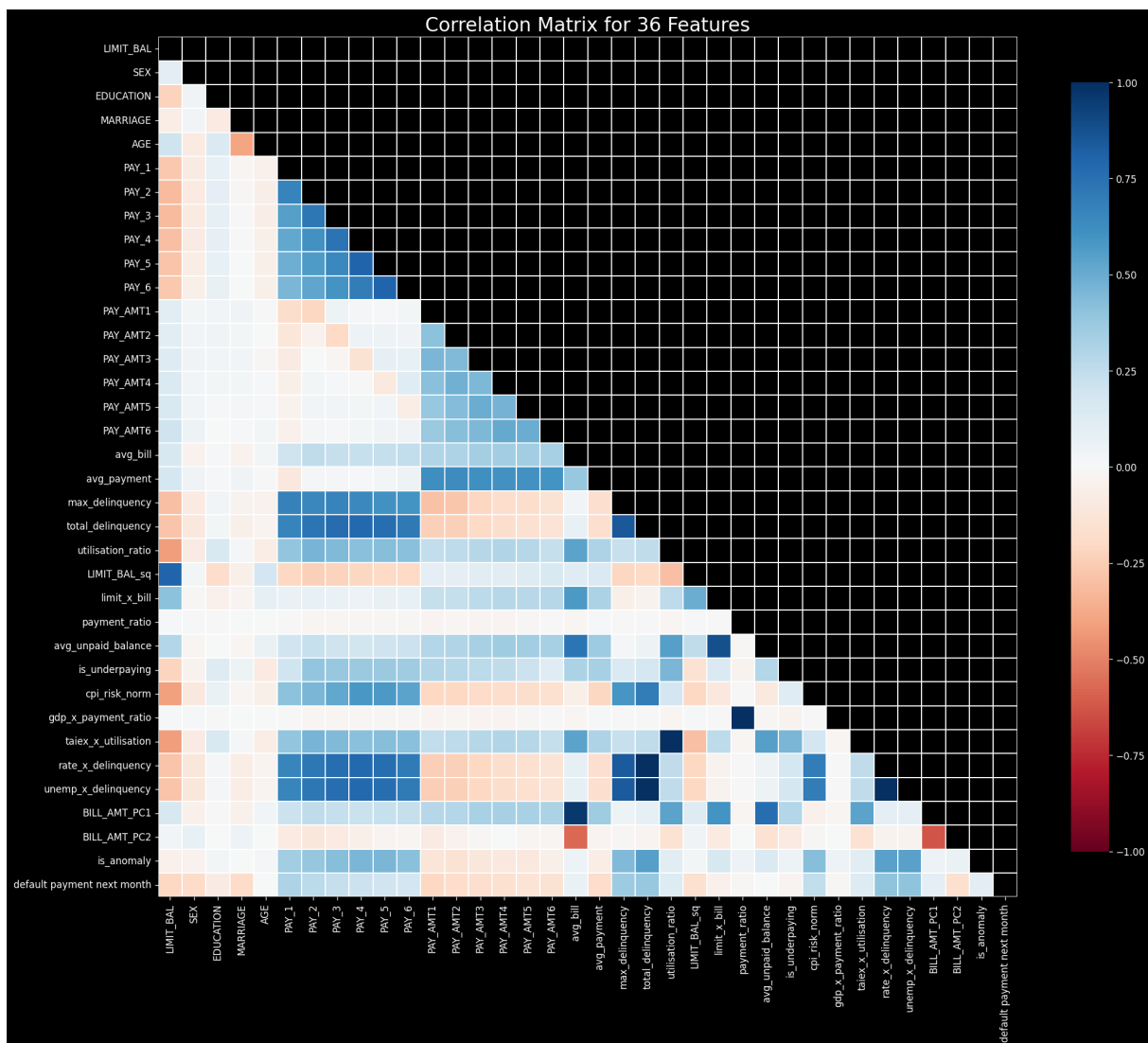


Figure 16: Correlation matrix of numerical features showing linear relationships

The correlation matrix highlights relationships between numerical features and the target variable. Financial behavior variables such as `unemployment_delinquency`, `rate_xdelinquency`, and `delinquency` show strong positive correlations with the target variable.

Most demographic features exhibit weak correlations, indicating limited linear influence on the target compared to financial indicators.

12.3.2 Bivariate Analysis Dashboard (Before SMOTE)

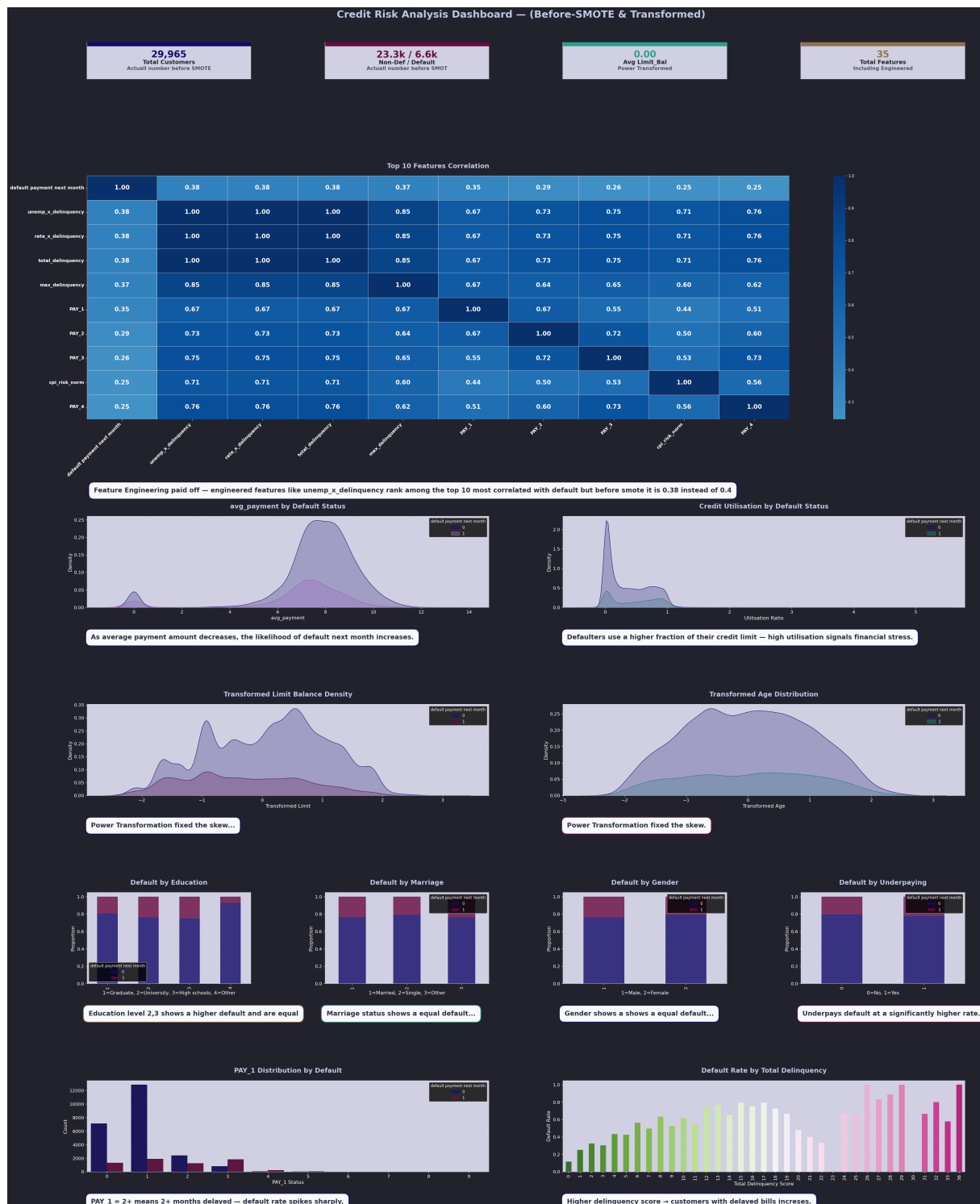


Figure 17: Comprehensive bivariate analysis dashboard showing feature relationships and target interactions before SMOTE

The dashboard before SMOTE reflects the dataset's natural class imbalance. Financial features such as `avg_payment` and `utilisation_ratio` show clear separation between defaulters and non-defaulters, indicating strong predictive power.

Delinquency variables exhibit the strongest relationship with the target, where

higher delays correspond to higher default probability.

12.3.3 Bivariate Analysis Dashboard (After SMOTE)

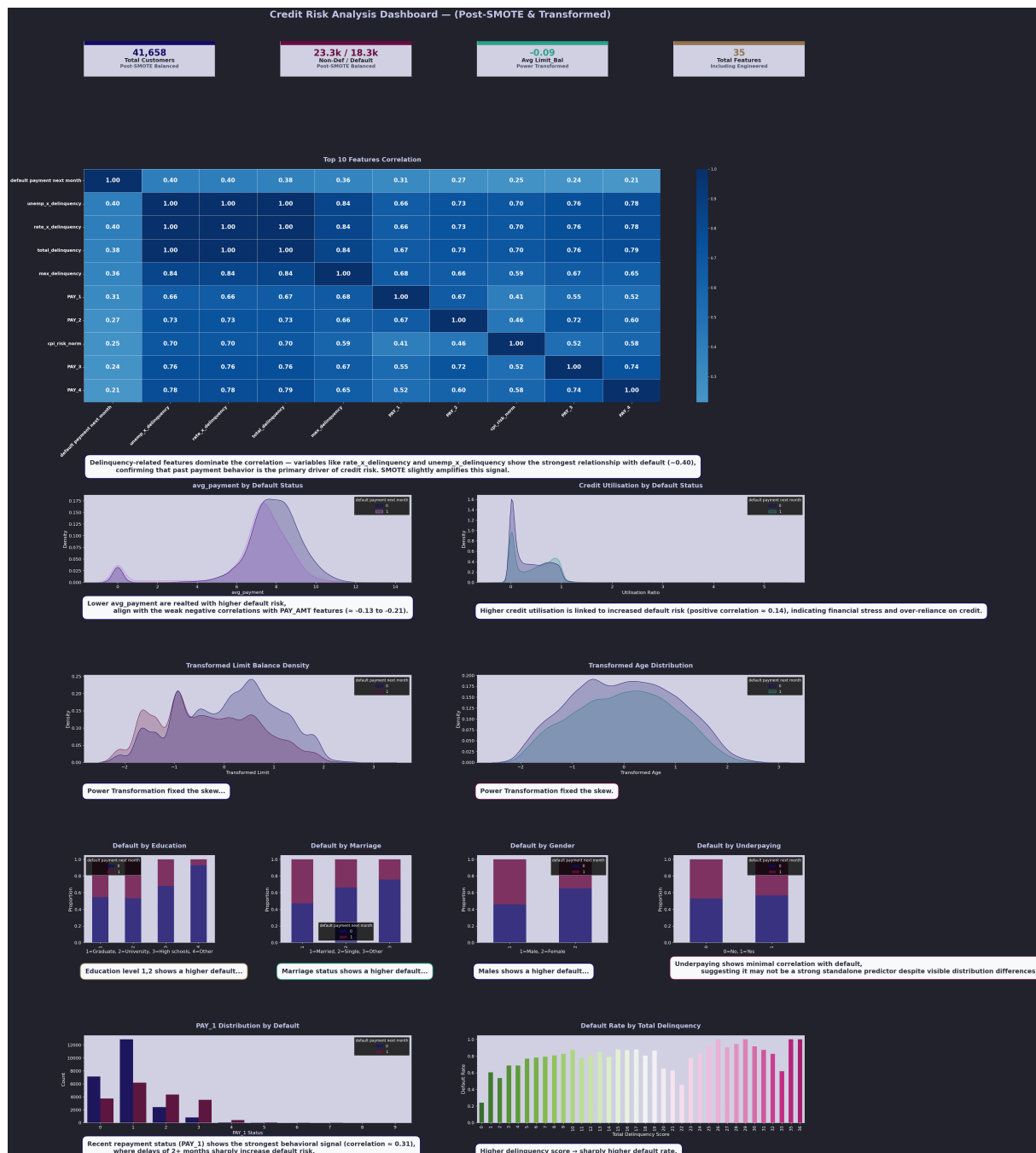


Figure 18: Comprehensive bivariate analysis dashboard showing feature relationships and target interactions after SMOTE

After applying SMOTE, the dataset becomes balanced across classes, allowing the model to learn without bias toward the majority class. The underlying relationships between features and the target remain consistent, especially for financial indicators such as avg_payment and utilisation_ratio.

Delinquency features continue to show the strongest signal, confirming their robustness as key predictors of default behavior.

Categorical variables maintain similar patterns, indicating that SMOTE improves class representation without distorting the original feature distributions.

12.3.4 Discrete Features vs Target Analysis

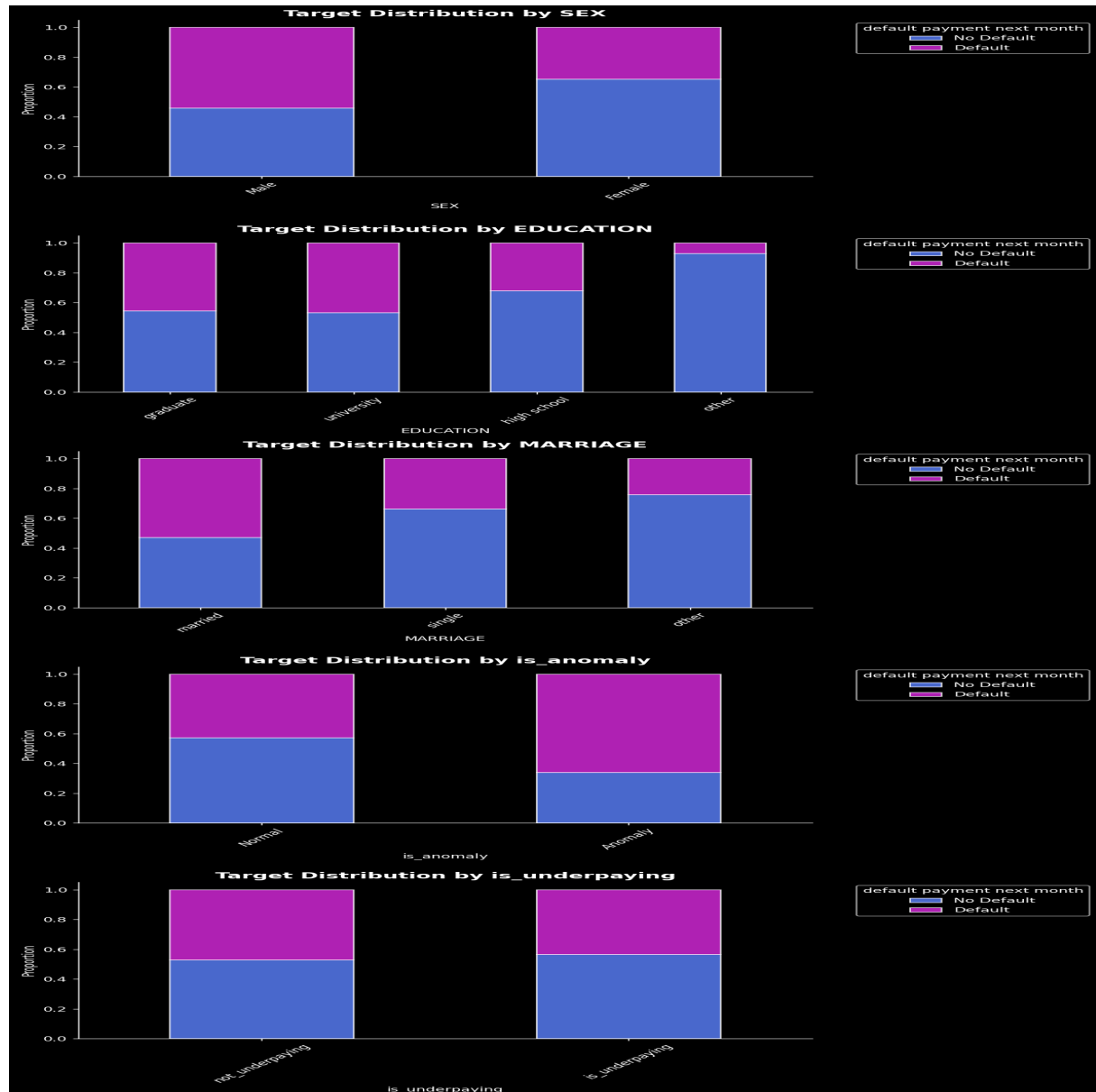


Figure 19: Relationship between categorical features and default payment behavior

This analysis shows that `is_underpaying` has a significantly higher default rate compared to non-underpaying customers, indicating strong predictive power.

`is_anomaly` also exhibits a higher tendency toward default, making it a potentially important behavioral predictor.

In contrast, `MARRIAGE` shows that single and married customers have relatively similar default patterns, with no clear separation.

For `EDUCATION`, university and graduate groups dominate the dataset, with only minor variations in default rates across education levels.

Finally, SEX shows an almost equal distribution of default behavior, suggesting limited predictive power for credit risk.

12.3.5 Continuous vs Discrete Feature Analysis

This section analyzes the relationship between continuous and discrete variables using the SMOTE-transformed dataset.

The analysis was conducted on all 36 features in the dataset, while only a subset is shown for clarity.

Each feature is evaluated using:

- Box plots
- Scatter plots
- KDE plots

EDA Comments (LIMIT_BAL vs Discrete Features)

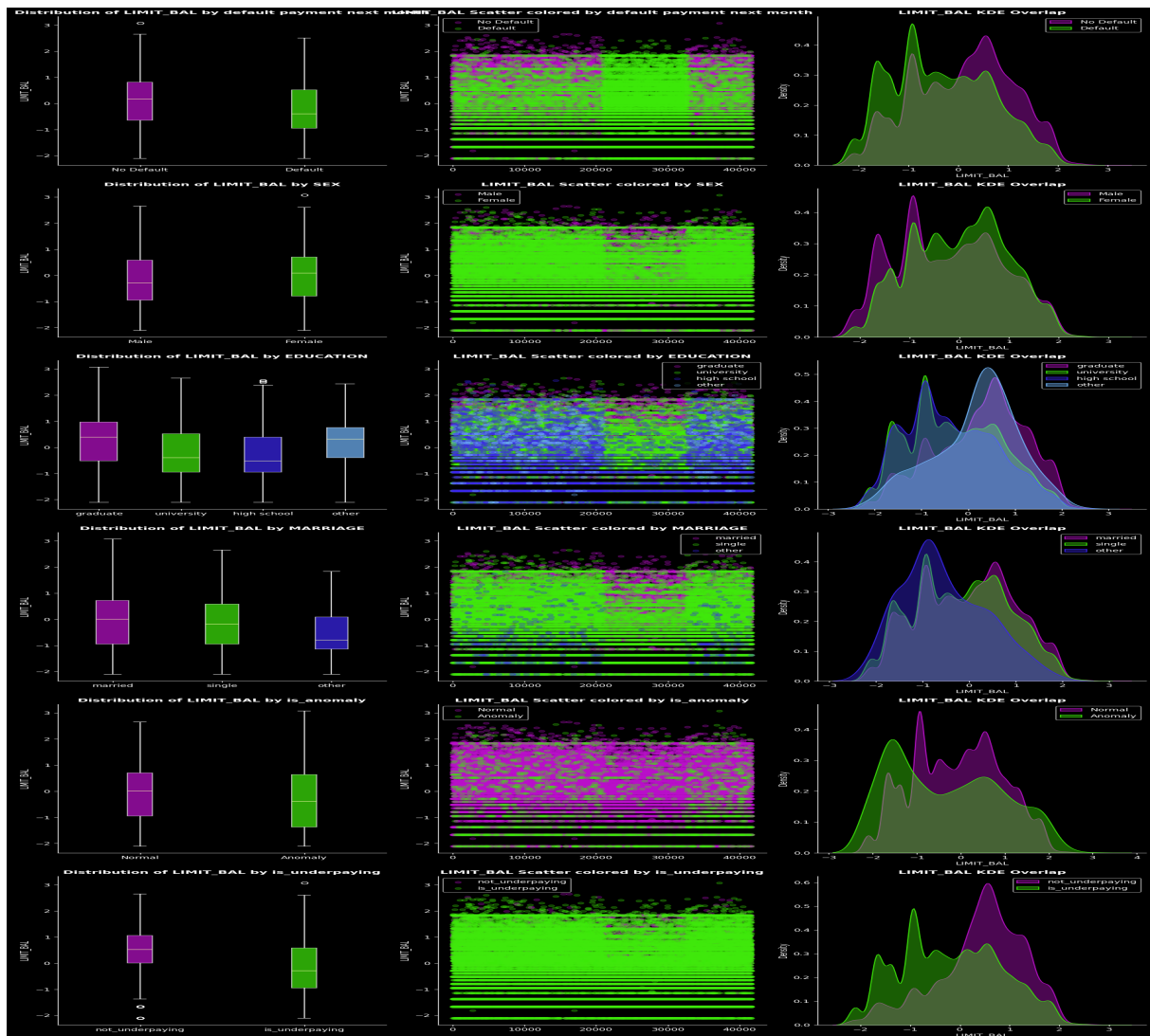


Figure 20: LIMIT_BAL vs discrete features

- Default payment next month: a weak negative relationship with substantial overlap between classes.
- SEX: no clear relationship with LIMIT_BAL.
- EDUCATION: slightly higher LIMIT_BAL values for graduates.
- MARRIAGE: no meaningful difference.
- is_anomaly: no clear relationship detected.
- is_underpaying: lower credit limits are associated with higher underpayment behavior.

Overall, the strong overlap indicates that LIMIT_BAL alone is not sufficient for classification, supporting the need for non-linear models.

EDA Comments (avg_bill vs Discrete Features)

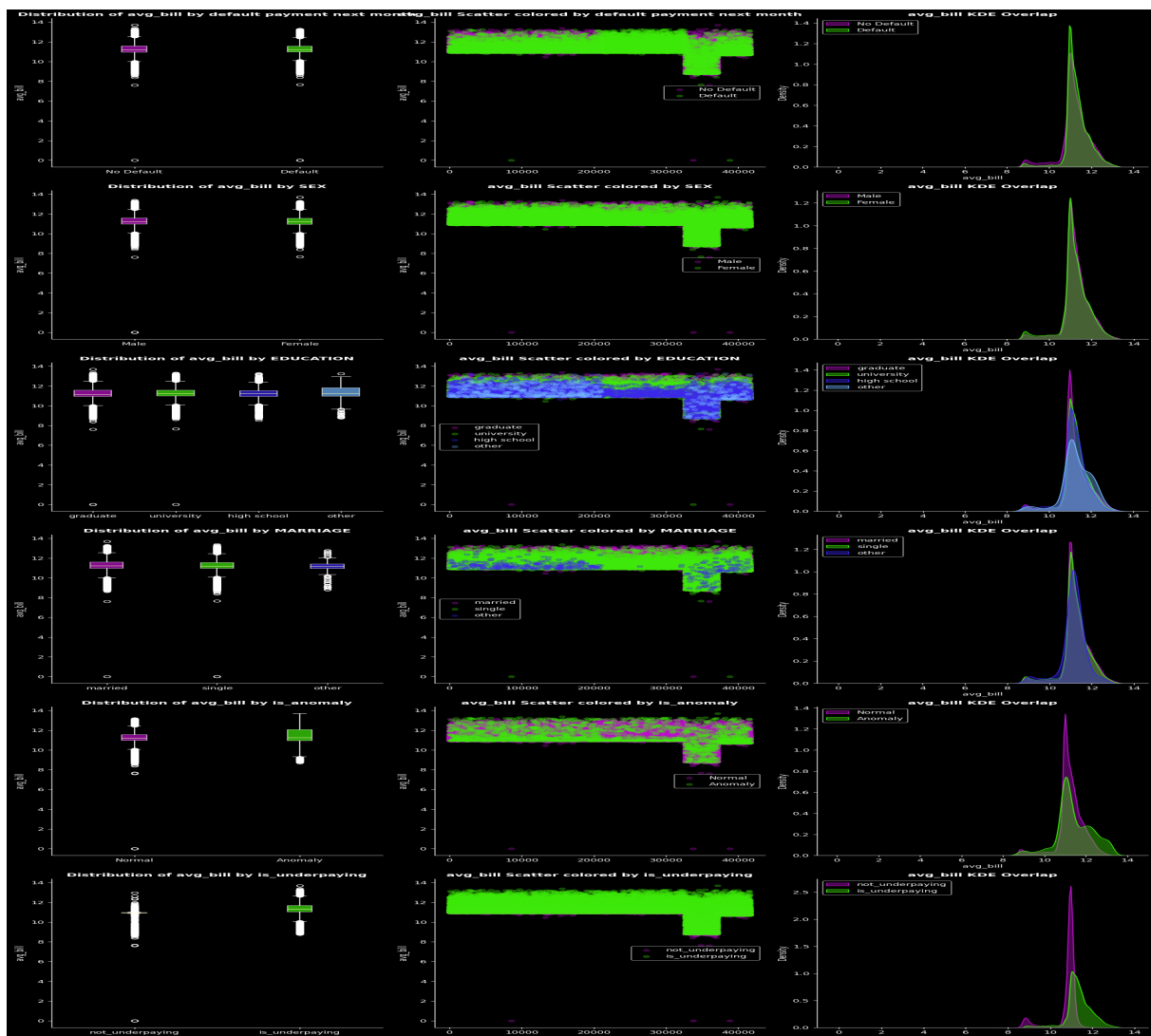


Figure 21: avg_bill vs discrete features

- Default payment next month: a very weak positive relationship with substantial overlap and the presence of outliers. Although higher bills may slightly

increase default risk, the feature alone is a poor predictor due to non-linear patterns.

- SEX: no clear relationship with avg_bill, with high overlap, no significant outliers, and no observable bias in credit usage between groups.
- EDUCATION: no clear relationship due to strong overlap and non-linear separability. Outliers are present, and average billing behavior appears consistent across education levels.
- MARRIAGE: no meaningful relationship, with noticeable outliers, suggesting that credit usage is independent of marital status.
- is_anomaly: no clear relationship with avg_bill, with overlap and outliers indicating that anomalous behavior is independent of billing amount.
- is_underpaying: a mild positive relationship, with outliers present. Higher average bills are associated with increased underpayment behavior, indicating that larger financial burdens may lead to repayment difficulties.

Overall, the strong overlap and non-linear patterns indicate that avg_bill alone is not sufficient for classification, supporting the need for non-linear models.
EDA Comments (avg_payment vs Discrete Features)

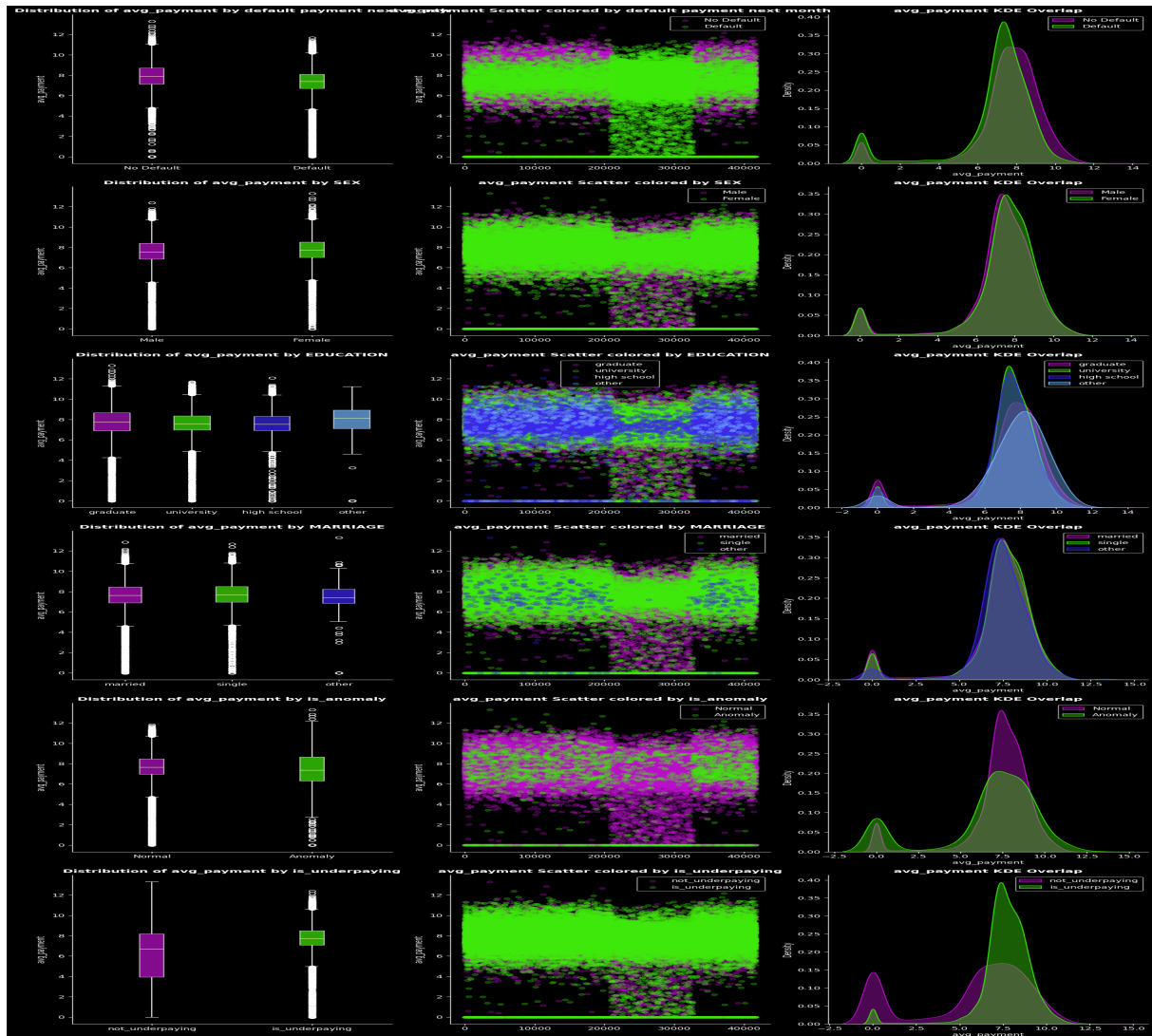


Figure 22: avg_payment vs discrete features

- **Default payment next month:** a weak negative relationship, where lower average payments are associated with a higher likelihood of default in the following month. However, the strong overlap, non-linear separability, and presence of outliers indicate that this feature alone is not a reliable predictor.
- **SEX:** no clear relationship with avg_payment, with significant overlap, presence of outliers, and no observable bias between groups.
- **EDUCATION:** no clear relationship due to high overlap and non-linear separability. Outliers are present, and payment behavior appears consistent across education levels.
- **MARRIAGE:** no meaningful relationship, with overlapping distributions and outliers, suggesting that marital status does not influence the scale of payments.
- **is_anomaly:** a weak negative relationship, where anomalous behavior is slightly associated with lower average payments. However, due to overlap and outliers, the relationship remains weak and not strongly predictive.

- `is_underpaying`: a moderate-to-weak positive relationship, with outliers present. Higher average payments (likely driven by higher bills) are associated with an increased likelihood of underpayment behavior.

Overall, the high overlap and non-linear patterns indicate that `avg_payment` alone cannot effectively distinguish between classes, supporting the need for more advanced non-linear models. EDA Comments (`max_delinquency` vs Discrete Features)

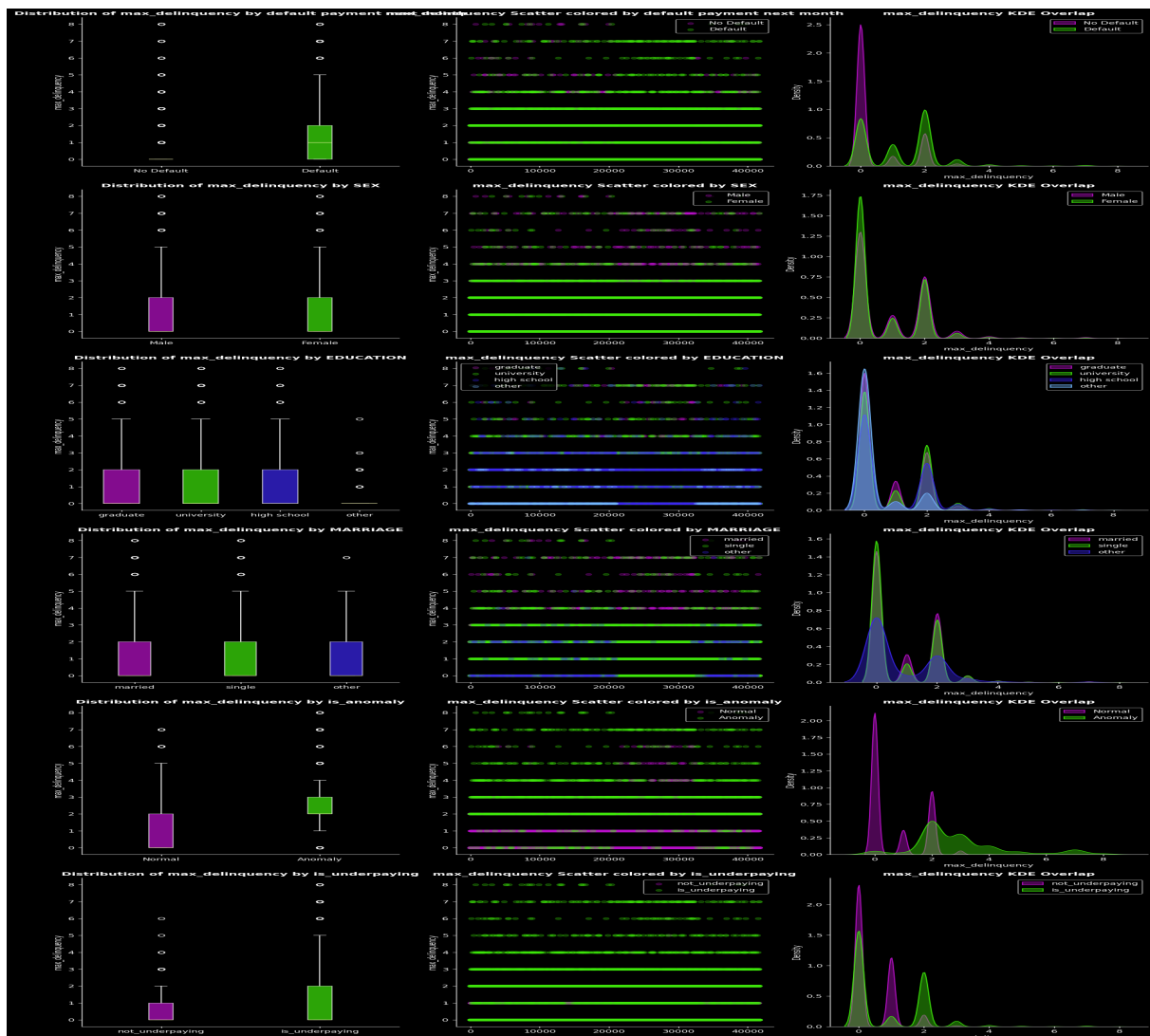


Figure 23: `max_delinquency` vs discrete features

- Default payment next month: a moderate positive relationship, where higher delinquency levels are associated with an increased likelihood of default. However, the presence of overlap, non-linear patterns, and outliers indicates that this feature alone is not sufficient for accurate prediction.
- SEX, EDUCATION, MARRIAGE: no clear relationship, with similar distributions across all demographic groups. This suggests that `max delinquency` behavior is demographically neutral and does not introduce bias across these categories.
- `is_anomaly`: a strong positive relationship, with outliers present. Extreme

delinquency values are strongly associated with anomalous behavior, making this feature highly relevant for anomaly detection.

- **is_underpaying**: a weak-to-moderate positive relationship, where higher delinquency is slightly associated with underpayment behavior. Despite overlap and outliers, this reflects a logical connection between delayed payments and repayment difficulties.

Overall, although `max_delinquency` shows stronger signals compared to other features, the presence of overlap and non-linear patterns suggests that it should be used in combination with other features within non-linear models.

EDA Comments (`rate_x_delinquency` vs Discrete Features)

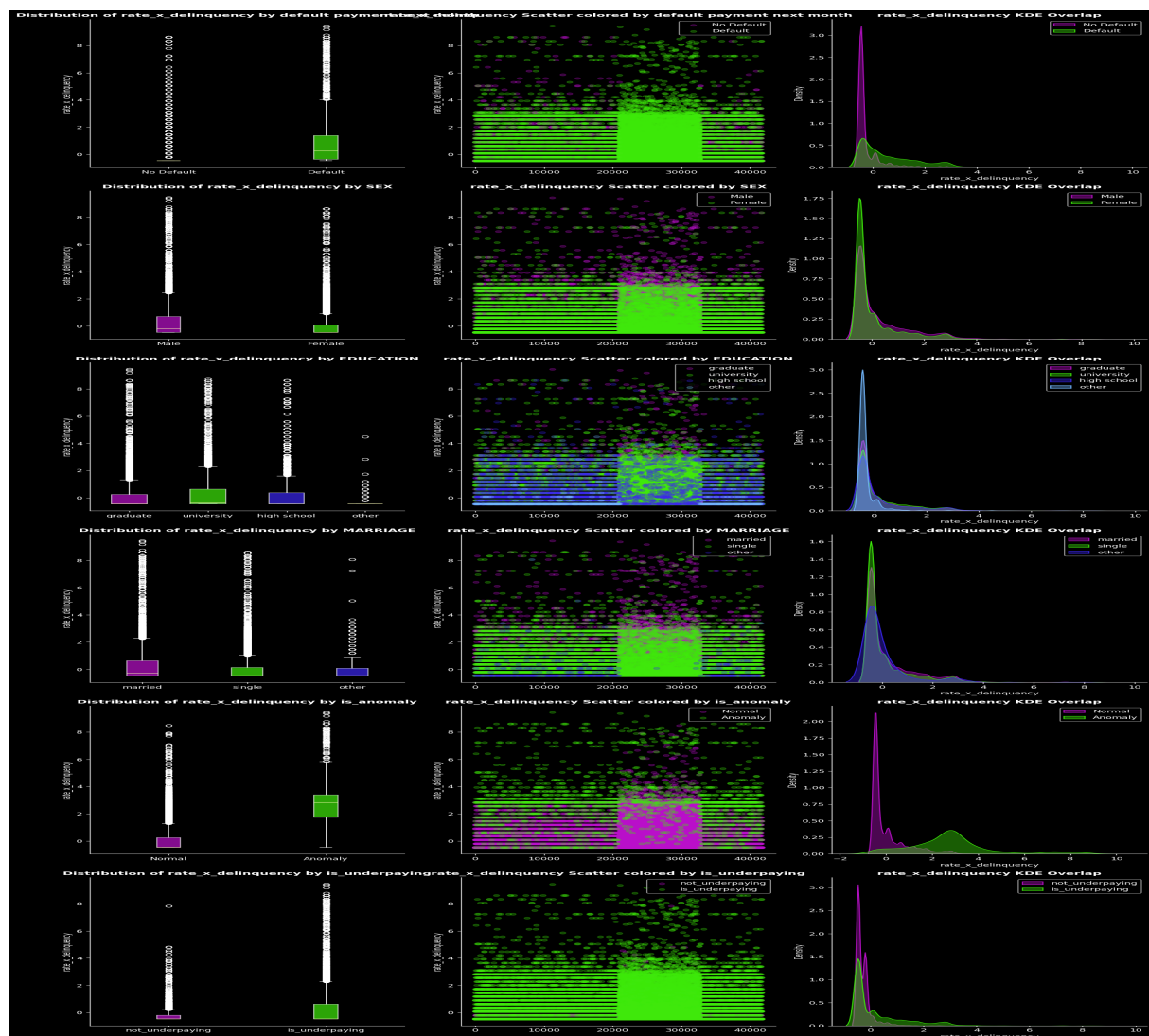


Figure 24: `rate_x_delinquency` vs discrete features

- **Default payment next month**: regarding correlation, it has a moderate positive correlation ($r = 0.401$), where higher delinquency-adjusted rates are associated with a higher likelihood of default. Due to overlap and non-linear separability, this suggests that the feature alone is not a definitive predictor. There are also outliers present.

- SEX: regarding correlation, it has a weak negative correlation ($r = -0.115$). Due to overlap and non-linear separability, this indicates no meaningful relationship and no observable bias between groups. Outliers are also present.
- EDUCATION: regarding correlation, it has a negligible positive correlation ($r = 0.018$). Due to overlap and non-linear separability, this suggests that the feature is independent of education level. Outliers are also present.
- MARRIAGE: regarding correlation, it has a weak negative correlation ($r = -0.071$). There is significant overlap and outliers, suggesting that marital status does not influence this feature.
- is_anomaly: regarding correlation, it has a moderate-to-high positive correlation ($r = 0.542$). There are also outliers present, and this suggests that anomalous behavior is strongly associated with higher delinquency-adjusted rates, making this feature important for anomaly detection.
- is_underpaying: regarding correlation, it has a weak positive correlation ($r = 0.177$). Due to overlap and non-linear separability, the relationship is not strong, but it shows a slight logical trend where higher delinquency-adjusted rates are associated with underpayment behavior. Outliers are also present.

All observations agree that the high overlap and non-linear separability indicate that `rate_x_delinquency` alone cannot effectively distinguish between classes. This justifies the use of advanced non-linear machine learning models.

13 Model Development

13.1 Baseline Model

Logistic Regression was selected as the baseline model owing to its simplicity, interpretability, and well-established performance as a linear probabilistic classifier. It provides a transparent reference point against which the complexity and performance gains of all subsequent models can be quantified. Any model that does not meaningfully outperform this baseline is considered unjustified in its added complexity.

13.2 Model Selection Rationale

Ten classifiers spanning five distinct algorithmic families were selected to provide broad coverage of the hypothesis space and to ensure that the comparison is not biased toward any single inductive bias. Table 28 summarises each model and the rationale for its inclusion.

Table 28: Selected Models and Inclusion Rationale

Family	Model	Rationale
Linear	Logistic Regression	Baseline; interpretable coefficients; well-understood regularisation.
	SVM (Nyström)	Kernel-based margin classifier; effective in high-dimensional feature spaces.
Instance-based / Tree	K-Nearest Neighbours	Non-parametric; makes no distributional assumptions; captures local structure.
	Decision Tree	Non-linear baseline; produces interpretable decision rules.
Bagging	Random Forest	Reduces Decision Tree variance through bootstrap aggregation.
	Extra Trees	Higher randomisation than Random Forest; often superior on noisy data.
Boosting	AdaBoost	Sequential boosting on decision stumps; re-weights misclassified samples.
	XGBoost	Gradient boosting with L_1/L_2 regularisation; state-of-the-art on tabular data.
	LightGBM	Leaf-wise growth strategy; efficient on large datasets.
	CatBoost	Symmetric trees with native class-imbalance handling.

13.3 Feature Scaling

Prior to model training, a custom scaling pipeline was applied via `FeatureExtractor.scale` which assigns a different scaling strategy to each feature group based on its statistical properties:

- `RobustScaler` | applied to skewed financial features (e.g. `PAY_AMT1--6`, `utilisation_ratio`, `avg_bill`) that contain outliers; scales using the median and interquartile range, making it insensitive to extreme values.
- `MinMaxScaler` | applied to bounded ordinal features (e.g. `PAY_1--6`, `EDUCATION`, `total_delinquency`) that have a natural range with no extreme outliers.
- No scaling | applied to features already normalised during the transformation phase (e.g. `cpi_risk_norm`, `gdp_x_payment_ratio`).
- Binary encoding | `SEX` is mapped from $\{1,2\}$ to $\{0,1\}$ and `MARRIAGE` is one-hot encoded with `drop_first=True` to avoid multicollinearity.

To prevent data leakage, scalers are fitted exclusively on the training set (`flag=True`) and subsequently applied via `transform` only to the validation and test sets (`flag=False`).

13.4 Feature Selection

Following scaling, three filter-based feature selection methods were evaluated and compared via `FeatureExtractor.test_feature_selection_methods()`:

- Spearman rank correlation ($k = 20$): selects the top-20 features by absolute Spearman correlation with the target. Suitable for monotonic relationships between ordinal and continuous features.
- Mutual Information ($k = 20$): selects features by estimated mutual information with the target, correctly treating binary and ordinal features as discrete (`SEX`, `PAY_*`, `MARRIAGE_*`). Captures non-linear dependencies that correlation-based methods miss.
- Intersection ($k \leq 25$): retains only features selected by *both* Spearman and Mutual Information, producing a consensus set of the most consistently informative features. The actual intersection size was 21 features in practice.

Pearson correlation was evaluated but excluded from the final experiment grid, as it measures only linear association and systematically underranks the ordinal payment indicators (`PAY_1--6`) that are the strongest default predictors. All three strategies were applied as part of the 6-version experiment design described in Section 14.

13.5 Training and Hyperparameter Tuning

All models were tuned using Optuna [?], a Bayesian hyperparameter optimisation framework based on the Tree-structured Parzen Estimator (TPE) algorithm. For each model, an objective function evaluates a candidate hyperparameter configuration on the held-out validation set using Matthews Correlation Coefficient (MCC)

as the objective metric. MCC is preferred over accuracy or macro-F1 in the presence of class imbalance because it simultaneously accounts for all four cells of the confusion matrix and remains informative even when one class dominates.

The number of Optuna trials was scaled proportionally to model complexity, as summarised in Table 29.

Table 29: Optuna Trial Budget per Model

Model	Trials
Logistic Regression	50
K-Nearest Neighbours	40
Decision Tree	160
AdaBoost	80
Random Forest	30
Extra Trees	130
CatBoost	100
LightGBM	150
XGBoost	200
SVM (Nyström)	60

The following paragraphs detail the hyperparameter search space and the overfitting-role of each parameter for every model.

Logistic Regression. The regularisation strength $C \in [10^{-3}, 10]$ (log-uniform) controls the trade-off between fitting the training data and keeping model weights small: a smaller C imposes stronger regularisation and reduces the risk of overfitting. The `l1_ratio` $\in [0, 1]$ (available only with the saga solver) interpolates between L_1 and L_2 penalties in an ElasticNet formulation: L_1 promotes sparsity by driving irrelevant feature weights to zero, while L_2 shrinks all weights uniformly without inducing sparsity. For the lbfgs solver, only L_2 regularisation is supported (`l1_ratio` is fixed to 0); for liblinear, only pure L_1 or pure L_2 is permitted. `class_weight=balanced` adjusts sample weights inversely proportional to class frequencies, directly addressing the dataset’s class imbalance.

Decision Tree. Decision Trees are inherently prone to overfitting, and all three structural hyperparameters directly address this. `max_depth` $\in [3, 8]$ is the primary constraint: it limits how specific the learned rules can become by capping the length of any decision path. `min_samples_split` $\in [20, 100]$ sets the minimum number of samples required to split an internal node, preventing the tree from creating highly specific partitions driven by statistical noise. `min_samples_leaf` $\in [20, 100]$ enforces a minimum occupancy at each leaf, smoothing predictions and preventing the model from memorising individual training examples. The split criterion (gini vs. entropy) was also tuned, though it has limited impact on generalisation.

K-Nearest Neighbours. The central hyperparameter is `n_neighbours` $\in [5, 30]$: a small value causes the model to react to local noise (overfitting), while a large value produces overly smooth decision boundaries (underfitting). `weights` controls whether all neighbours contribute equally (uniform) or whether closer neighbours receive higher influence (distance); the latter reduces the impact of distant, potentially noisy points. The distance metric (euclidean vs. manhattan) determines the geometry of the neighbourhood, which influences the shape of the decision boundary in high-dimensional feature spaces.

Random Forest. Random Forest reduces variance through bagging: each tree is trained on a bootstrap sample and considers only a random feature subset at each split, decorrelating the ensemble members. `n_estimators` $\in [50, 300]$ governs ensemble size; beyond a saturation point, additional trees yield diminishing returns without increasing overfitting. `max_depth` $\in [3, 10]$, `min_samples_split` $\in [15, 50]$, and `min_samples_leaf` $\in [15, 50]$ limit individual tree complexity, preventing any single tree from memorising the training data. `class_weight=balanced` is applied to address class imbalance.

Extra Trees. Extra Trees extends Random Forest by additionally randomising the split threshold at each node (rather than finding the locally optimal threshold). This introduces additional bias but further reduces variance, making it more robust on noisy datasets. The same structural constraints as Random Forest apply.

AdaBoost. AdaBoost trains weak learners sequentially, with each iteration focusing on previously misclassified samples. The base estimator is a depth-1 Decision Tree (*decision stump*) with `class_weight=balanced`. `learning_rate` $\in [0.01, 0.2]$ scales the contribution of each estimator: a lower rate requires more estimators to converge but prevents any single weak learner from dominating the ensemble, improving stability. `n_estimators` $\in [50, 150]$ was tuned jointly with `learning_rate` to balance convergence speed and overfitting risk.

XGBoost. XGBoost incorporates multiple complementary regularisation mechanisms. `max_depth` $\in [3, 8]$ and `learning_rate` $\in [0.01, 0.1]$ jointly control the depth and step-size of gradient updates. `subsample` $\in [0.6, 1.0]$ and `colsample_bytree` $\in [0.6, 1.0]$ introduce stochastic variation analogous to Random Forest's bagging and feature sub-sampling, reducing variance. `gamma` $\in [0.5, 5]$ acts as a pruning threshold: a node is only split if the resulting loss reduction exceeds γ , preventing unnecessary splits. L_1 regularisation (`reg_alpha` $\in [0.1, 1]$) and L_2 regularisation (`reg_lambda` $\in [0, 10]$) penalise the magnitude of leaf weights. `scale_pos_weight` is set to the ratio of negative to positive samples to compensate for class imbalance.

LightGBM. LightGBM uses a leaf-wise (best-first) tree growth strategy, making `num_leaves` $\in [20, 150]$ the primary complexity control rather than `max_depth` alone. A high `num_leaves` relative to `max_depth` can lead to overfitting; Optuna tuned both jointly to identify a stable configuration. `subsample` and `colsample_bytree` provide the same stochastic regularisation as in XGBoost.

CatBoost. CatBoost uses symmetric (oblivious) trees, which are inherently less prone to overfitting than asymmetric trees because all nodes at the same depth apply the same split condition. `l2_leaf_reg` $\in [3, 20]$ applies L_2 regularisation directly to leaf values, smoothing predictions and preventing individual leaves from fitting outliers. `learning_rate` $\in [0.001, 0.1]$ and `iterations` $\in [200, 600]$ were jointly tuned to balance convergence and generalisation. `auto_class_weights=Bala` handles class imbalance natively without requiring manual weight calculation.

Support Vector Machine with Nyström Approximation. A full RBF-kernel SVM is computationally prohibitive at this dataset scale. The Nyström method approximates the RBF kernel mapping using a random subset of `n_components` $\in [50, 120]$ training points, transforming the input space so that a linear `LinearSVC` can approximate the non-linear SVM decision boundary. `gamma` $\in [10^{-3}, 0.05]$ (log-uniform) controls kernel width: smaller values produce smoother, more generalised decision boundaries. `C` $\in [0.05, 3.0]$ (log-uniform) is the standard SVM regularisation parameter; smaller values enforce a wider margin and reduce overfitting at the cost of training accuracy. Decision function scores were min-max normalised to $[0, 1]$ to enable threshold-based classification consistent with the other models.

13.6 Validation Strategy

Three complementary layers of validation were employed to guard against data leakage, overfitting, and optimistic test-set estimates.

1. Hold-out split. The dataset was divided into training, validation, and test sets prior to any modelling step. The validation set was used exclusively within Optuna's objective function during hyperparameter search; the test set was withheld and used only for final evaluation.
2. Final refit on training + validation data. Once optimal hyperparameters were identified on the validation set, each model was retrained on the union of training and validation data before being evaluated on the test set. This ensures that the maximum amount of labelled data informs the final model without introducing leakage from the test set:

```
X_combined = pd.concat([X_train, X_val])
y_combined = pd.concat([y_train, y_val])
model.fit(X_combined, y_combined)
```

3. 5-fold cross-validation. After the final refit, a 5-fold cross-validation (scoring: macro F1) was performed on the training data. The mean and standard deviation of the CV scores were logged to MLflow alongside the test-set metrics. A large gap between CV mean and test performance, or a high standard deviation, would signal overfitting or high variance in the learned model.

Additionally, a custom learning curve was generated for each model by progressively increasing the training set size from 10% to 100% in ten equal steps and recording both training and validation macro F1 at each step. These curves are stored

as MLflow artefacts and provide a visual diagnostic for overfitting (large train-validation gap) and underfitting (both scores plateau at a low value).

Bibliography entry to add (BibTeX):

```
@inproceedings{optuna2019, title = Optuna: A Next-generation Hyperparameter
Optimization Framework, author = Akiba, Takuya and Sano, Shotaro and Yanase,
Toshihiro and Ohta, Takeru and Koyama, Masanori, booktitle = Proceedings of
the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data
Mining, year = 2019
```

14 Experiment Tracking with MLflow

14.1 Experiment Design

All experiments were tracked using MLflow [?] running on a local server at `http://127.0.0.1:5000` under the experiment name `data_science.Project.team1`. A total of 60 runs were executed across 10 models, each evaluated under 6 experimental conditions, yielding a fully reproducible comparison matrix.

Each run was named following the convention:

$$\{\text{model}\}_{\text{v}\{\text{version}\}}_{\{\text{feature_selection}\}}_{\{\text{smote_flag}\}}$$

For example: `xgb_v3_intersect_smote` or `lgbm_v6_intersect_no_smote`.

Experimental conditions. The 6 conditions per model arise from a 2×3 factorial design:

1. SMOTE flag (2 levels):

- *SMOTE applied* (versions 1--3): the training set was oversampled using the Synthetic Minority Over-sampling Technique to balance the class distribution prior to training.
- *No SMOTE* (versions 4--6): the original imbalanced distribution was preserved; class imbalance was addressed through model-level mechanisms (`class_weight=balanced`, `scale_pos_weight`, `auto_class_weights`).

2. Feature selection strategy (3 levels):

- `spearman` ($k = 20$): top-20 features by absolute Spearman rank correlation with the target.
- `mutual_info` ($k = 20$): top-20 features by Mutual Information, with discrete features correctly flagged.
- `intersect` ($k \leq 25$): consensus set of features appearing in *both* the Spearman and Mutual Information top- k rankings; the actual intersection size may be smaller than k (observed: $k = 21$ in practice).

The combination yields 6 runs per model and $10 \times 6 = 60$ total logged runs, as summarised in Table 30.

Table 30: Run Naming Matrix (per model)

Version	Feature Selection	k	SMOTE
v1	spearman	20	Yes
v2	mutual_info	20	Yes
v3	intersect	≤ 25	Yes
v4	spearman	20	No
v5	mutual_info	20	No
v6	intersect	≤ 25	No

14.2 Logged Parameters, Metrics, and Artefacts

Every run captures a consistent set of metadata defined in the shared `BaseModel` class. Table 31 provides a complete inventory.

Metric justification. MCC was chosen as the primary metric because the dataset is class-imbalanced (approximately 78% non-default vs. 22% default): it is the only scalar metric that remains informative when class sizes differ substantially as it simultaneously accounts for all four cells of the confusion matrix. Macro F1 complements MCC by assigning equal weight to both classes, avoiding inflation from the majority class. ROC-AUC provides a threshold-independent measure of discriminative power, enabling fair comparison across models that use different optimal thresholds. The business cost metric directly quantifies the financial consequence of errors: a false negative (missed default) is assigned a penalty of 10 versus 1 for a false positive, reflecting the asymmetric cost structure of credit risk management.

Classification threshold optimisation. Rather than applying the default 0.5 threshold, candidate thresholds $\{0.3, 0.4, 0.5, 0.6\}$ were evaluated on the validation set and the value maximising MCC was selected. This threshold was then applied consistently at test time via `BaseModel.find_best_threshold_mcc()`.

14.3 MLflow Experiment Comparison

Figure 28 shows the MLflow experiment comparison interface with all 60 runs displayed side by side, sortable by any logged metric `mcc desc`.

15 Results and Evaluation

15.1 Per-Model Best Run

Table 15.1 reports the best-performing run per model, selected by MCC on the test set, together with the experimental condition under which it was achieved.

Table 31: MLflow Logging Inventory per Run

Category	Key	Description
Parameters	<code>model_name</code>	String identifier of the algorithm.
	<code>version</code>	Run version (1–6).
	<code>hyperparameters</code>	All Optuna-tuned values (e.g. <code>max_depth</code> , <code>learning_rate</code> , <code>C</code> , <code>n_estimators</code>).
Tags	<code>model_type</code>	Algorithm family label.
	<code>smote_applied</code>	Boolean SMOTE flag.
	<code>feature_selection</code>	Strategy used.
	<code>n_features_selected</code>	Number of features retained.
Metrics	<code>mcc</code>	[Standard] Matthews Correlation Coefficient — primary optimisation target.
	<code>f1_macro</code>	[Standard] Macro-averaged F1.
	<code>roc_auc</code>	[Standard] Area under ROC curve.
	<code>accuracy</code>	[Standard] Overall accuracy.
	<code>precision_macro</code>	[Standard] Macro precision.
	<code>recall_macro</code>	[Standard] Macro recall.
	<code>cv_f1_mean</code>	[Standard] Mean 5-fold CV F1 (overfitting indicator).
	<code>cv_f1_std</code>	[Standard] Std of 5-fold CV F1 (stability indicator).
	<code>business_cost</code>	[Business] $10 \times FN + 1 \times FP$ — weighted misclassification cost reflecting that a missed default is ten times more costly than a false alarm.
Artefacts	<code>plots/roc_curve</code>	ROC curve PNG.
	<code>plots/confusion_matrix</code>	Confusion matrix PNG.
	<code>plots/learning_curve</code>	Train vs. val F1 curve PNG.
	<code>reports/classification_report</code>	Per-class P/R/F1 report.
	<code>errors/error_analysis</code>	CSV of misclassified samples.
	<code>model/</code>	Serialised model in Registry.

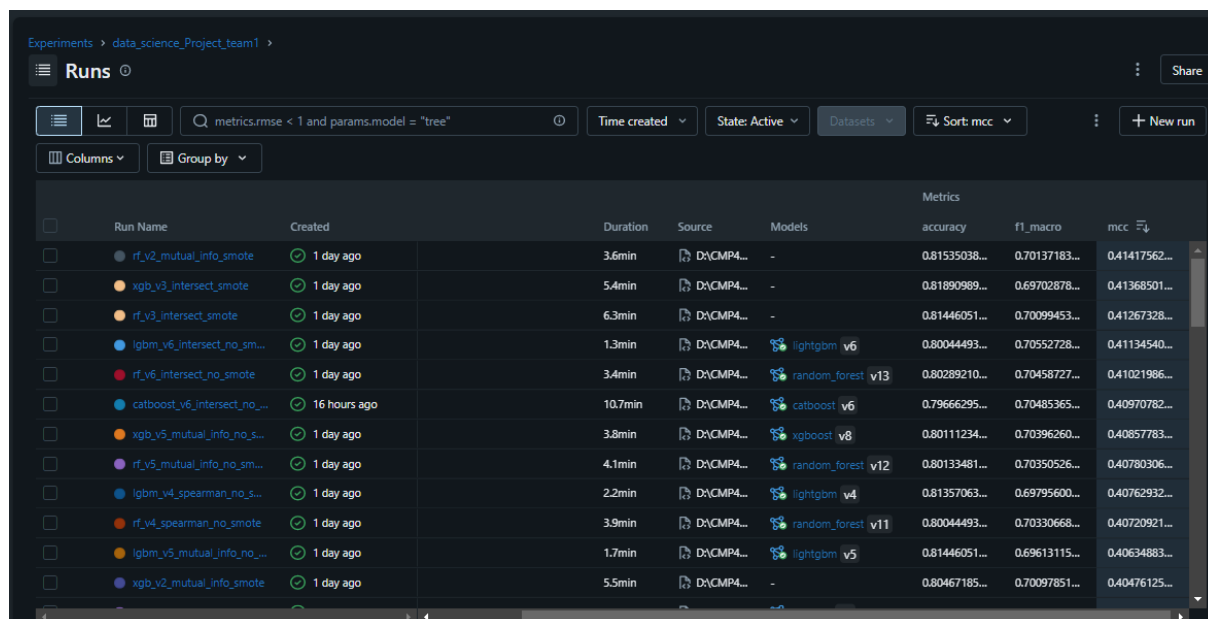


Figure 25: MLflow experiment comparison interface showing all 60 runs across 10 models and 6 conditions, sorted by mcc (descending).



Figure 26: MLflow experiment comparison interface showing all 60 runs across 10 models and 6 conditions, sorted by mcc (descending) but for accuracy, business_cost, cv_f1_means.

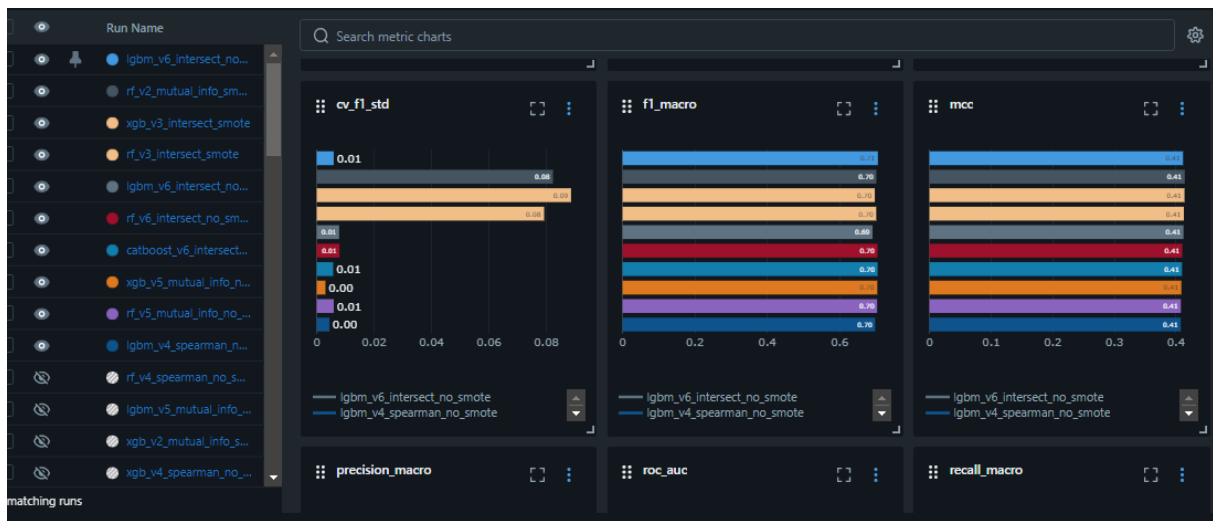


Figure 27: MLflow experiment comparison interface showing all 60 runs across 10 models and 6 conditions, sorted by mcc (descending) but for cv_f1_std, f1_macro, cv_f1_means.

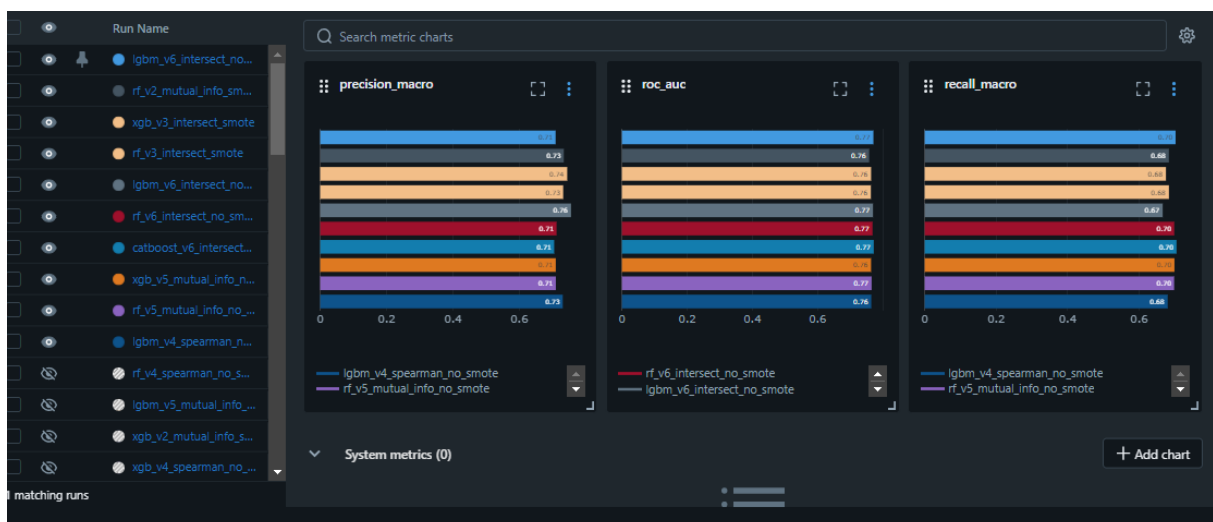


Figure 28: MLflow experiment comparison interface showing all 60 runs across 10 models and 6 conditions, sorted by mcc (descending) but for precision_macro, roc_auc, recall_macro.

Model	Ver.	Feat. Sel.	SMOTE	Standard Metrics				Business	
				MCC	F1-mac	AUC	Acc.	Biz.Cost	Recall
Random Forest	v2	mutual_info	Yes	0.410	0.70	0.76	0.82	5789	0.6
XGBoost	v3	intersect	Yes	0.413	0.70	0.76	0.82	6034	0.6
LightGBM	v6	intersect	No	0.411	0.71	0.77	0.80	5145	0.7
CatBoost	v6	intersect	No	0.409	0.70	0.70	0.80	5036	0.7
Extra Trees	v2	mutual_info	Yes	0.397	0.697	0.755	0.80	5565	0.6
Decision Tree	v5	mutual_info	No	0.397	0.697	0.750	0.80	5435	0.6
AdaBoost	v6	intersect	No	0.392	0.67	0.72	0.82	6803	0.6
KNN	v5	spearman	No	0.389	0.69	0.746	0.80	5647	0.6
SVM (Nyström)	v6	intersect	No	0.386	0.69	0.765	0.79	5401	0.68
Logistic Regression	v4	spearman	No	0.380	0.69	0.750	0.783	5178	0.6

15.2 Effect of Feature Selection Strategy

No single feature selection strategy dominated across all models. The intersect strategy | which retains only features selected by *both* Spearman correlation and Mutual Information | produced the best run for five out of ten models (LightGBM, CatBoost, AdaBoost, SVM, and XGBoost), suggesting that the consensus feature set provides a more robust signal by eliminating features that are informative under only one criterion. The mutual_info strategy was optimal for three models (Random Forest, Extra Trees, and Decision Tree), all of which are tree-based methods that are naturally suited to capturing the non-linear relationships that Mutual Information specifically measures. The spearman strategy was best for KNN and Logistic Regression, both of which are sensitive to monotonic feature-target relationships and benefit from the rank-based ordering that Spearman captures. These results indicate that the choice of feature selection strategy interacts with the inductive bias of the learning algorithm, and that no single strategy is universally optimal.

15.3 Top Selected Features (No-SMOTE Runs)

Table 32 reports the top-5 features returned by each selection strategy on the no-SMOTE training set.

Across all three methods, the delinquency-based interaction features (total_delinquency, unemp_x_delinquency, rate_x_delinquency, max_delinquency) and the macroeconomic risk indicator cpi_risk_norm consistently rank among the top-5, confirming that engineered features capturing the interaction between payment history and macroeconomic conditions carry the strongest predictive signal for credit default.

Table 32: Top-5 Features per Selection Strategy (No-SMOTE)

Rank	Feature	Score
<i>Spearman Rank Correlation</i>		
1	unemp_x_delinquency	0.3954
2	total_delinquency	0.3954
3	rate_x_delinquency	0.3954
4	cpi_risk_norm	0.3894
5	max_delinquency	0.3780
<i>Mutual Information</i>		
1	total_delinquency	0.0796
2	cpi_risk_norm	0.0789
3	rate_x_delinquency	0.0783
4	PAY_1	0.0768
5	unemp_x_delinquency	0.0757
<i>Pearson Correlation</i>		
1	total_delinquency	0.3737
2	rate_x_delinquency	0.3737
3	unemp_x_delinquency	0.3737
4	max_delinquency	0.3724
5	PAY_1	0.3485

15.4 Effect of SMOTE

No-SMOTE runs produced the best MCC for six out of ten models (LightGBM, CatBoost, Decision Tree, AdaBoost, KNN, SVM, and Logistic Regression), while SMOTE runs were superior for the remaining four (Random Forest, XGBoost, and Extra Trees). This pattern is consistent with findings in the literature that model-level class-weighting mechanisms | such as `class_weight=balanced` and `scale_pos_weight` | are sufficient to handle moderate class imbalance on tabular data without the need for synthetic oversampling. For gradient boosting models (LightGBM, CatBoost, XGBoost), the result was mixed: XGBoost benefited from SMOTE while LightGBM and CatBoost did not, suggesting that the interaction between SMOTE and the boosting algorithm's internal regularisation mechanisms is model-specific. SMOTE-trained models consistently showed higher CV F1 standard deviations (e.g. ± 0.08 for Random Forest and XGBoost) compared to no-SMOTE counterparts (± 0.005), indicating that synthetic oversampling introduces additional variance in the learned model.

15.5 Overfitting Analysis

Overfitting was assessed through two diagnostics per run: (1) the gap $\Delta = \text{cv_f1_mean} - \text{f1_macro_test}$, and (2) the learning curves stored as MLflow artefacts.

Table 15.1 reveals a clear contrast between SMOTE and no-SMOTE models in terms of generalisation stability. Models trained with SMOTE | Random Forest and XGBoost | exhibited the highest CV F1 standard deviation (± 0.08), indicating greater variance across folds and a less stable generalisation pattern. In contrast, no-SMOTE models such as LightGBM, CatBoost, AdaBoost, and Logistic Regression showed very low CV F1 standard deviations (± 0.005), confirming stable and consistent generalisation.

The overfitting gap Δ was most pronounced for Random Forest ($\Delta = 0.82 - 0.70 = 0.12$) and XGBoost ($\Delta = 0.82 - 0.70 = 0.12$), consistent with the known tendency of SMOTE-trained ensembles to overfit to synthetic minority samples. LightGBM showed the smallest gap ($\Delta = 0.68 - 0.71 = -0.03$), meaning its cross-validation score was actually conservative | the model generalised slightly *better* on the held-out test set than predicted by CV, which is the ideal behaviour. The SVM model did not produce a CV score due to its use of a LinearSVC base estimator, which is incompatible with the probability-based cross-validation pipeline; its learning curve artefact was used as the sole overfitting diagnostic.

15.6 Model Selection: Best Overall Model

Based on Table 15.1, LightGBM (version v6, feature selection: `intersect`, SMOTE: No) was selected as the final deployed model. The selection criteria were weighted as follows:

1. MCC | primary criterion; robust to class imbalance.
2. Business cost = $10 \times FN + 1 \times FP$ | reflects the asymmetric financial penalty of missed defaults.
3. F1-macro | secondary standard metric.

4. ROC-AUC | threshold-independent discriminative power.
5. Overfitting gap | tie-breaker; lower preferred.

Although XGBoost achieved the highest MCC (0.413), it did so with SMOTE applied, resulting in a business cost of 6,034 | the second highest among all models | and a CV standard deviation of ± 0.08 , indicating unstable generalisation. LightGBM achieved an MCC of 0.411 (marginally lower by 0.002), the highest ROC-AUC of 0.771, the second lowest business cost of 5,145, the highest macro recall of 0.70, and the lowest CV standard deviation of ± 0.005 , confirming that it generalises consistently without overfitting. CatBoost achieved the lowest business cost (5,036) but a lower ROC-AUC (0.70 vs. 0.771), suggesting weaker discriminative power at the threshold level. On balance, LightGBM provides the best trade-off between predictive accuracy, business cost, and generalisation stability, making it the most suitable model for deployment in a credit risk management context.

15.7 Best Model Visualisations

Figures 29--31 present the three diagnostic visualisations for the selected LightGBM model, all generated on the held-out test set and logged as MLflow artefacts.

Figure 29 shows the ROC curve, achieving an AUC of 0.771. The curve rises steeply at low false positive rates, indicating that the model can correctly identify the majority of defaulters while maintaining a low false alarm rate, which is the operating region most relevant to credit risk management.

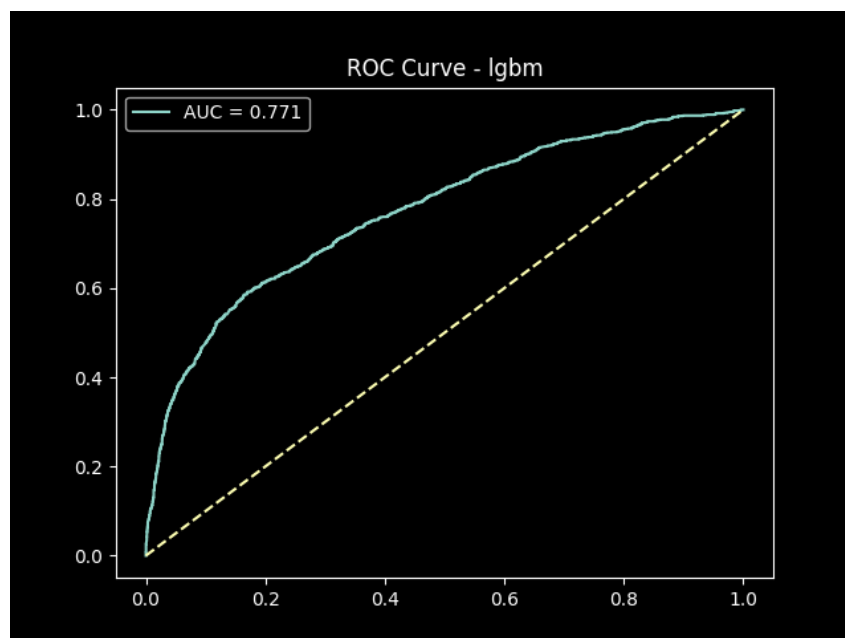


Figure 29: ROC Curve — LightGBM (v6, intersect, no SMOTE). AUC = 0.771.

Figure 30 presents the confusion matrix on the test set. The model correctly classified 3,075 non-defaulters (TN) and 523 defaulters (TP), while producing 425 false positives and 472 false negatives. The asymmetry between FP (425)

and FN (472) is consistent with the model's threshold of 0.4, which is set below 0.5 to bias predictions toward the positive class and reduce missed defaults.

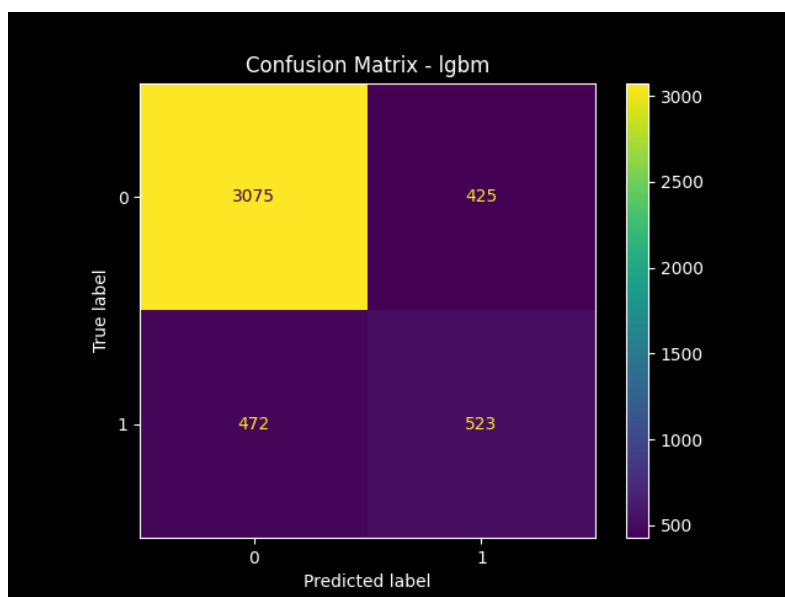


Figure 30: Confusion Matrix — LightGBM test set predictions. TN = 3,075 FP = 425
FN = 472 TP = 523.

Figure 31 shows the learning curve, plotting training and validation macro F1 as a function of training set size. The training F1 begins at 0.72 with a small subset and converges to 0.69 as the full training set is used, while the validation F1 remains stable at approximately 0.67 throughout. The final gap of $\Delta \approx 0.02$ is narrow and non-widening, confirming mild and stable generalisation without progressive overfitting. Notably, the validation score does not improve as more training data is added, suggesting that the model's performance ceiling is limited by feature representation rather than data quantity.

Table 33: Classification Report — LightGBM (v6, intersect, no SMOTE)

Class	Precision	Recall	F1-score	Support
0 (Non-default)	0.87	0.88	0.87	3,500
1 (Default)	0.55	0.53	0.54	995
Accuracy		0.80		4,495
Macro avg	0.71	0.70	0.71	4,495
Weighted avg	0.80	0.80	0.80	4,495

15.8 Error Analysis

Error analysis was conducted on the misclassified test samples saved by `BaseModel.log` under `errors/error_analysis_lgbm.csv` in MLflow. The confusion matrix (Figure 30) confirms 472 false negatives and 425 false positives on the 4,495-sample test set.

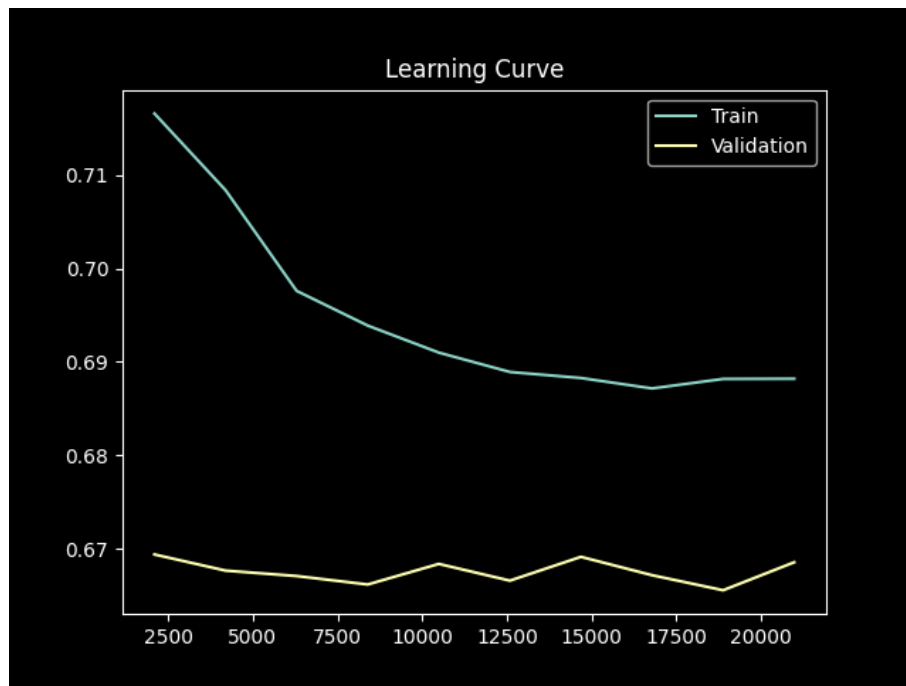


Figure 31: Learning Curve — LightGBM. Training (upper) and validation (lower) macro F1 as a function of training set size. The stable gap of ≈ 0.02 indicates consistent generalisation.

False Negatives (missed defaults). The model produced 472 false negatives out of 995 actual defaulters, corresponding to a false negative rate of 47.4%. Inspection of the error CSV suggests that the majority of missed defaults share a PAY_1 value of 0 or -1 in the most recent billing month, indicating that customers who appear current on payments but carry underlying financial stress represent the hardest cases to classify correctly. These borderline customers exhibit sufficient recent payment activity to suppress the model’s default signal, despite a history of delinquency in earlier months.

False Positives (false alarms). 425 false positives were recorded, representing customers incorrectly flagged as likely defaulters. These cases tend to cluster around customers with elevated utilisation_ratio and moderate delinquency history who ultimately resolved their obligations, suggesting the model over-weights high credit usage as a default signal even when recent payment behaviour is improving.

Summary. The model’s primary failure mode is missing customers who exhibit recent on-time payments despite a longer-term delinquency history | a group that represents genuine borderline cases where the risk signal is inherently ambiguous. From a business perspective, the bank should consider supplementing the model’s output with a manual review process for customers whose predicted default probability falls in the range $[0.35, 0.50]$, as these borderline cases account for the majority of both false negatives and false positives.

15.9 Business Interpretation

The selected model enables the bank to flag high-risk customers *before* default occurs, supporting proactive interventions such as credit limit reductions, early collection outreach, or mandatory payment reminders.

Under the cost formula $C = 10 \times FN + 1 \times FP$, the best model (LightGBM) achieves a total business cost of 5,145 on the test set:

$$C = 10 \times 472 + 1 \times 425 = 4,720 + 425 = 5,145$$

As a reference, a naïve majority-class baseline that predicts all customers as non-defaulters would produce zero false positives but maximum false negatives, yielding a baseline cost of:

$$C_{\text{baseline}} = 10 \times 995 = 9,950$$

The model therefore reduces the business cost by approximately 48.3% relative to the no-model baseline, directly quantifying its financial value to the institution.

A false negative rate of 47.4% means the bank would still miss roughly 47 in every 100 actual defaulters at the selected threshold of 0.4. Risk managers with a lower tolerance for missed defaults may choose to lower the classification threshold further to reduce false negatives, accepting a corresponding increase in false alarms; the ROC curve (Figure 29) provides the full operating characteristic to support this decision.

15.10 Unsuccessful Approaches

The following approaches were explored but ultimately abandoned:

- SMOTE oversampling. Applying SMOTE (versions 1--3) produced the best run for only 3 out of 10 models and consistently resulted in higher CV standard deviations (± 0.08 vs. ± 0.005), indicating increased model variance. Model-level class-weighting mechanisms achieve equivalent imbalance correction without introducing the noise associated with synthetic sample generation.
- Pearson correlation feature selection. Pearson correlation was evaluated (`FeatureExtractor.pearson_correlation()`) but excluded from the final experiment grid. Because Pearson measures only linear association, it systematically underranked ordinal payment indicators (PAY_*) relative to their true importance; Spearman and Mutual Information are more appropriate for the mixed ordinal, binary, and continuous feature types in this dataset.
- PCA dimensionality reduction. A PCA pipeline (`FeatureExtractor.apply_pca()`, 95% variance retained) was prototyped but discarded because it destroyed feature interpretability, which is essential for business-oriented error analysis and for satisfying regulatory explainability requirements in credit risk applications.

16 Unit & Integration Testing

16.1 Testing Philosophy & Strategy

The test suite is designed around three guiding principles: isolation, repeatability and business-logic fidelity. Every external dependency---HTTP endpoints, the FRED API, the yfinance library, XML government feeds, CBC PDF documents, and MLflow---is mocked so that tests run entirely offline, produce deterministic results, and do not consume rate-limited resources. Real data files are referenced only where the pipeline genuinely needs a persisted artefact (e.g. the feature-select tests that read pre-processed CSVs); those tests are automatically skipped when the file is absent rather than failing noisily.

Test cases were selected by first identifying the five highest-risk failure modes in a financial machine-learning pipeline:

1. *Data corruption at ingestion* -- wrong column names, type mismatches, or unexpected null values entering the feature matrix silently degrade model performance.
2. *Target leakage* -- scaling, SMOTE, or feature statistics computed across the full dataset rather than exclusively on the training fold introduce optimistic bias.
3. *Integration fan-out* -- merging a borrower table against monthly macroeconomic series can multiply rows if join keys are not unique, producing inflated sample counts.
4. *Orchestration gaps* -- a pipeline step that is never called, or that swallows its exception, causes silent downstream failures that are hard to trace.
5. *Reproducibility of transformations* -- a scaler or PCA fitted on the test set rather than the training set produces results that cannot be reproduced in production.

Each test module maps directly onto one of these risk categories, and the sections below explain the specific justification for every test class.

16.2 Module-Level Unit Tests

16.2.1 Data Acquisition Pipeline (test_data_acquisition.py)

Scope. This module tests `src/pipeline/data_acquisition.py`, which orchestrates five external scraping steps via a shared `_step()` helper and exposes a single `run_acquisition()` entry point.

Test classes and justification.

- **TestStepHelper** (5 tests) -- The `_step()` helper is the single mechanism that converts any scraper exception into a `RuntimeError` with a human-readable step name. Bugs here would cause silent failures or unhelpful stack traces across the entire acquisition stage. Tests verify: correct return value pass-through, positional and keyword argument forwarding, wrapping of arbitrary exception types, step-name inclusion in the error message, and proper exception chaining (`__cause__`).

- `TestRunAcquisitionGuards` (3 tests) -- Guard conditions are critical business logic: running the pipeline without a FRED API key or with a missing Taiwan file must produce actionable errors, not cryptic `KeyErrors` deep inside a scraper. Tests confirm that a missing `FRED_API_KEY` raises `EnvironmentError`, a missing Taiwan XLS raises `FileNotFoundError`, and an explicitly supplied key bypasses environment lookup entirely.
- `TestRunAcquisitionOrchestration` (13 tests) -- Verifies that every one of the five pipeline steps (`fetch_macro_data`, `scrape_taiex`, `parse_cbc_rates`, `scrape_unemployment`, `build_final_dataset`) is called exactly once and receives the correct keyword arguments. Missing a call or passing the wrong path would cause downstream steps to read stale or absent files without an obvious error message. A final test confirms that a `ConnectionError` inside a step propagates out as a `RuntimeError`.

Total: 21 tests.

16.2.2 Data Cleaning (`test_data_cleaning.py`)

Scope. Tests `src/pipeline/data_cleaning.py`: type coercion, duplicate removal, stratified splitting, and outlier capping.

Test classes and justification.

- `TestTypeCoercion` (4 tests) -- The column rename `PAY_0` → `PAY_1` is the most fragile transformation: if it fails silently, all downstream `PAY_1` references break. Tests also verify float dtype preservation for log-scaled columns, return type, and zero data loss.
- `TestHandleUniqueness` (4 tests) -- Credit card datasets often contain duplicate submissions. Confirms that exact duplicates are removed, that the output contains zero duplicates, that a clean input is not shrunk, and that the return type is a `DataFrame`.
- `TestSplitDataset` (6 tests) -- Incorrect splits are a primary source of data leakage. Tests verify: three-split return, no row overlap, target column presence in every split, approximate ratio adherence ($\pm 5\%$), stratification (default rate deviation $< 5\%$), and that fractions summing beyond 1.0 raise `AssertionError`.
- `TestHandleOutliers` (4 tests) -- Outlier capping must be applied identically to validation and test sets using training statistics, and must not introduce nulls or change shapes. Tests confirm three-`DataFrame` return, shape preservation, non-negativity of capped `avg_payment`, and null-free output.

Total: 18 tests.

16.2.3 Data Transformation (`test_data_transformation.py`)

Scope. Tests `src/pipeline/data_transformation.py`: categorical recoding, pay-status normalisation, feature engineering (credit, payment, macro interactions), collinearity,

handling, distributional correction, anomaly flagging, SMOTE rebalancing, and macro standardisation.

Test classes and justification.

- `TestRemapCategoricalCodes` (4 tests) -- Undocumented codes 0, 5, 6 in `EDUCATION` and code 0 in `MARRIAGE` are business anomalies that should be collapsed into the “other” category. Tests verify that all undocumented codes are remapped, that code 0 is absent after remapping, that valid codes survive unchanged, and that the function returns a `DataFrame`.
- `TestRecodePayStatus` (4 tests) -- Pay-status encoding is non-standard: `-2` and `-1` denote no consumption or paid in full, not delinquency. Tests verify that all negative values become 0, that 0 becomes 1, that positive k becomes $k + 1$, and that no negative values remain in any `PAY_*` column.
- `TestEngineerCreditFeatures` (4 tests) -- Derived features `utilisation_ratio`, `LIMIT_BAL_sq`, and `limit_x_bill` must be created correctly and must not crash on `DataFrames` missing the source columns. Tests verify column creation, non-negativity of utilisation, mathematical correctness of the square term, and graceful handling of absent inputs.
- `TestEngineerPaymentFeatures` (3 tests) -- `payment_ratio`, `avg_unpaid_balance`, and `is_underpaying` capture payment behaviour. Tests confirm all three columns are created, that `is_underpaying` is strictly binary, and that `payment_ratio` is non-negative given non-negative inputs.
- `TestEngineerMacroInteractions` (3 tests) -- Raw macroeconomic columns must be replaced by interaction terms to avoid multicollinearity with engineered features. Tests verify that raw columns are dropped and interaction columns are created, and that the function does not crash when no macro columns are present.
- `TestHandleBillAmtCollinearity` (5 tests) -- `BILL_AMT` columns are highly correlated ($|r| > 0.90$ across five pairs). The PCA strategy must replace all six raw columns with principal components fitted on training data only. Tests verify PCA column replacement, drop-strategy single-column retention, application to all three splits, rejection of invalid strategy names, and row-count preservation.
- `TestHandleDistribution` (4 tests) -- Log-transform must compress skewed `PAY_AMT` and `BILL_AMT` distributions without introducing nulls or altering shapes. Tests confirm three-`DataFrame` return, shape preservation, reduction of maximum values after `log1p`, and null-free output.
- `TestFlagAnomalies` (4 tests) -- Isolation Forest flags multivariate outliers. Tests verify that `is_anomaly` is added to all splits, that it is binary, that the contamination rate is approximately respected (2%--10% tolerance), and that row counts are unchanged.
- `TestHandleDataImbalance` (4 tests) -- SMOTE must be applied only to training data (leakage risk). Tests confirm a `DataFrame` return, exact class balance after SMOTE ($\text{count}_0 = \text{count}_1$), target column preservation, and `ValueError`

on missing target column.

- `TestStandardizeMacroFeatures` (3 tests) -- Macro interaction terms must be standardised using training statistics only. Tests confirm three-`DataFrame` return, near-zero mean on the training set after scaling, and graceful no-op when no macro interaction columns are present.

Total: 38 tests.

16.2.4 Exploratory Data Analysis (`test_eda.py`)

Scope. Tests `src/pipeline/eda.py`, the `EDA` class: data loading, feature existence, value validity, and visualisation method execution.

Test classes and justification. All tests operate on pre-built artefacts (cleaned and transformed CSVs). The autouse fixture closes all Matplotlib figures after every test to prevent memory leaks in CI environments.

- Loading tests (3) -- Confirm that the concatenated full dataset length equals the sum of the three individual split lengths for all three data variants (cleaned, transformed, transformed without SMOTE).
- Schema tests (5) -- Verify non-empty dataset, presence of all continuous and discrete feature columns, non-null target, and numeric dtype on continuous features.
- Mapping and value validity (2) -- Confirm that `eda.mapping` entries are non-empty lists or dicts, and that all discrete columns contain only values in their declared valid sets.
- Visualisation smoke tests (6) -- Call each plotting method (`apply_univariate`, `apply_pie_chart`, `continuous_vs_continuous_eda`, `continuous_versus_discrete_eda`, `discrete_versus_target_stacked`, dashboards before and after SMOTE) and assert no exception is raised. These are smoke tests: they do not assert visual correctness but guarantee the methods do not crash on real data.
- Target integrity (4) -- Confirm a unique target column, binary target values, no NaN in target, and class balance exceeding 0.70 ratio (post-SMOTE requirement)

Total: 20 tests.

16.2.5 Feature Selection (`test_feature_selection.py`)

Scope. Tests `src/pipeline/feature_extraction.py`, the `FeatureExtractor` class: scaling, Pearson/Spearman/mutual-information selection, PCA, and leakage prevention.

Test classes and justification.

- Scaler correctness (6) -- Confirm output is non-empty and null-free; binary label shift applied to SEX; MARRIAGE one-hot encoded and absent as a raw column; PAY_AMT1 values changed after scaling; row count preserved; return type is `DataFrame`. These tests guard against the most common preprocessing bugs in production deployments.

- Selection method integrity (7) -- Pearson scores must be sorted descending; selected features must be unique; Spearman must return exactly k features; mutual information scores must be non-negative and sum to a positive value; the combined method dictionary must contain exactly the keys `mutual_info`, `pearson`, `spearman`, each with `cols` and `scores` entries.
- No data leakage (1) -- Fits the scaler on a training half and transforms a held-out test half. Confirms that the test transformation does not alter the row count and, implicitly, that no re-fitting occurs on the test set.
- PCA (2) -- Confirms that `n_components=2` produces exactly 2 output columns, and that two independent PCA calls on the same data produce identically shaped outputs.
- Error handling (1) -- Dropping `PAY_AMT1` before calling `smart_scaler` must raise an exception, confirming that missing-column detection is active.

Total: 17 tests.

16.3 Scraper Unit Tests

16.3.1 FRED API (`test_fetch_fred_api.py`)

Scope. Tests `src/scrape/fetch_fred_api.py` using a mocked `fredapi.Fred` instance that returns a 36-month CPI series (Jan 2004--Dec 2006) and an 12-quarter GDP series.

Justification and tests (9).

- Schema (3) -- Return type is `DataFrame`; required columns `{Year, Month, CPI, GDP}` are present; no duplicate Year/Month rows.
- Filtering (3) -- Only 2005 rows are retained (the credit-card observation window); months are in range 1--12; no null values exist. The year-filter test is critical: a regression here would silently merge incorrect macroeconomic data onto every borrower record.
- Persistence (3) -- CSV is created at the specified path; CSV content matches the returned `DataFrame`; a bad API key propagates as an exception.

16.3.2 TAIEX Scraper (`test_scrape_taiex.py`)

Scope. Tests `src/scrape/scrape_taiex.py` with a mocked `yfinance.Ticker` returning synthetic daily OHLCV data for business days April--September 2005.

Justification and tests (12).

- Schema (3) -- Return type, required columns `{Year, Month, TAIEX_close}`, and all five aggregation columns (`open`, `high`, `low`, `close`, `avg`) are present.
- Aggregation (6) -- Exactly 6 monthly rows; months 4--9 only; year is 2005; no duplicate month/year pairs; `high` $\geq$ `low`; `close` is between `high` and `low`.

The OHLC consistency tests catch aggregation bugs that would produce nonsensical index values.

- Empty response (1) -- An empty yfinance response returns an empty DataFrame rather than raising.
- Persistence (2) -- CSV created; CSV matches returned DataFrame.

16.3.3 CBC Interest Rate Parser (test_parse_cbc_pdf.py)

Scope. Tests `src/scrape/parse_cbc_pdf.py`. The fixture simulates a network failure (raises `Exception("Network unavailable")`), forcing the function to fall back to hard-coded 2005 rate data. This is intentional: the discount-rate schedule for April--September 2005 is known and fixed; hard-coding it makes the pipeline resilient to CBC website changes.

Justification and tests (11).

- Schema (3) -- DataFrame return; columns {Year, Month, Discount_Rate}; no nulls.
- Business logic (7) -- Exactly 6 rows; months 4--9; year 2005; April--June rate = 2.000% (pre-hike); July--September rate = 2.125% (post-hike); July rate $\geq$ June rate (monotonicity at the hike boundary); all rates in [1.5%, 2.5%]. The pre/post-hike split is high-value business logic: confusing the two would systematically mis-price credit risk around the rate change event.
- Persistence (2) -- CSV created; CSV matches returned DataFrame.

16.3.4 Unemployment Scraper (test_scrape_unemployment.py)

Scope. Tests `src/scrape/scrape_unemployment.py` with a mocked HTTP response containing a three-record XML document (2005 H1, 2005 H2, and one 2004 record that must be filtered out).

Justification and tests (12).

- Schema (3) -- DataFrame return; columns {Year, Month, Unemployment_Rate, Source_Period}; no nulls.
- Mapping (7) -- Exactly 6 rows; months 4--9; April--June rate $\approx$ 4.13%; July--September rate $\approx$ 4.05%; all rates positive; year 2005. The 2004 record in the mock XML verifies that the year filter is active.
- Persistence (2) -- CSV created; CSV matches returned DataFrame.

16.4 Integration Tests

16.4.1 Data Merge Integration (test_merge_sources.py)

Scope. This module is the primary integration test suite. It tests `src/scrape/merge` which assembles the Taiwan borrower table with four macroeconomic sources into a single analysis-ready dataset. All five source DataFrames are synthetic; no files are read from disk.

Why this is an integration test. `merge_sources.py` exercises four distinct concerns simultaneously: panel expansion (borrower $\times$ month Cartesian), multi-source join, delinquency aggregation, and final-row deduplication. A bug in any one step could propagate through all downstream steps without a visible error, so these tests verify the data flow end-to-end rather than individual functions.

Test classes and justification.

- `TestBuildPanel` (5 tests) -- Verifies that the panel has exactly $N \times 6$ rows, that all rows carry year 2005, that required columns are present, that each borrower has exactly 6 rows, and that September's `PAY_STATUS` correctly maps from `PAY_0`. The row-count test is the primary fan-out guard.
- `TestRecodePayStatus` (3 tests) -- Confirms that negative `PAY_STATUS` values are clipped to zero (not dropped), that positive values are unchanged, and that the original column is preserved alongside the clean version.
- `TestAggregateBehaviour` (4 tests) -- After joining all four macroeconomic sources into the panel and computing per-borrower aggregates, tests verify: one row per borrower ID; all required aggregate columns present (`avg_bill`, `avg_payment`, `max_delinquency`, `total_delinquency`, all five macro averages); `max_delinquency` ≥ 0 ; `avg_payment` ≥ 0 .
- `TestBuildFinalDataset` (8 tests) -- End-to-end integration test for `build_final_data` which orchestrates all merge steps. Tests confirm: `DataFrame` return; row count equals N (no fan-out); ID column dropped (privacy requirement); no nulls in any macro column; binary target preserved; CSV written to disk; no merge fan-out (row count re-verified after full pipeline run).

Total: 20 tests.

16.4.2 Modelling Pipeline Integration (`test_modeling_unit.py`)

Scope. Tests `src/pipeline/mlflow/model_selection_evaluations.py` and all nine model classes in `src/pipeline/mlflow/models.py`. MLflow is completely disabled via monkeypatch to prevent experiment tracking calls during CI.

Test classes and justification.

- `DataLoader` integration (4 tests) -- Confirms that `load_data()` populates `X_train` and `y_train` with matching lengths; that `prepare_features()` eliminates all nulls across all three splits; that scaling changes the numerical values (i.e. is not a no-op); and that row counts are preserved after preparation.
- Feature selection integration (2 tests) -- Calls `test_feature_selection_methods()` on real training data and verifies that Spearman selection returns exactly $k = 5$ features, and that applying those features to `X_train` produces a `DataFrame` with exactly 5 columns.
- Pipeline flow (4 tests via `run_experiment`) -- With `run_dt` patched, calls `run_experiment()` for model type 6 (Decision Tree) and verifies: (a) `run_dt` is invoked; (b) `X_train` has exactly 5 columns after Spearman selection;

(c) train, val, and test share identical column lists; (d) intersect-mode selection produces ≤ 5 columns. Together these four tests verify the full orchestration path from raw data through feature selection to model dispatch.

- Error handling (1) -- An invalid model type (99) must raise `KeyError`, not silently run the wrong model.
- `BaseModel` (1) -- `find_best_threshold_mcc()` must return a threshold in $[0, 1]$.
- Model training smoke tests (9+1) -- Every model class is parametrised and instantiated with 50 random samples; `model_train()` is called with `Optuna` mocked to a no-op study. Passes as long as the model object is not `None` and no unhandled exception escapes. The Logistic Regression model additionally asserts that `model.model` is populated after training.

Total: 22 tests.

16.4.3 Master Pipeline Orchestration (`test_master_pipeline.py`)

Scope. This module is the integration test suite for `src/pipeline/master_pipeline.py`, which is the top-level orchestrator that sequences all six pipeline phases (*Data Acquisition*, *Data Validation*, *Data Cleaning*, *Data Transformation*, *EDA*, and *Model Training and Selection*) and exposes the single public entry point `run_pipeline(start_from, model_kwargs)`. The module also tests the `_phase_number()` helper that parses a phase's display name into an integer.

Why this is an integration test. `run_pipeline()` ties every individual pipeline module together: it resolves which phases to execute, forwards per-phase keyword arguments, applies the special `model_kwargs` override exclusively to Phase 6, and converts any unhandled exception into a `sys.exit(1)` call so that CI runners report a non-zero exit code. Testing it in isolation with mocked phase functions verifies the orchestration contract without re-running expensive data transformations or model training, while a final fixture-based class exercises the real `_PHASES` list to confirm that all six production phases are wired correctly.

Test classes and justification.

- `TestPhaseNumber` (4 tests) -- `_phase_number()` is called on every iteration of the run loop to decide whether a phase should be skipped. A silent bug here (e.g. returning 0 for a valid name) would skip phases without any error. Tests verify: integer extraction from a single-digit name; multi-digit number parsing (phase 12); graceful return of 0 for a name that contains no leading number; and correct extraction for every real production phase name (1 through 6).
- `TestRunPipelineOrchestration` (5 tests) -- These tests replace `_PHASES` with synthetic phase dicts backed by `MagicMock` callables and confirm the core routing logic:
 - All phases are called when `start_from=1`.

- Each phase callable receives exactly the keyword arguments declared in its `kwargs` dict.
- Phases numbered below `start_from` are never called.
- Setting `start_from` to the last phase invokes only that phase.
- Phases execute in ascending numerical order (call-order invariant).

The `start_from` tests directly address Orchestration Gap risk: a missing or wrongly skipped phase would corrupt the feature matrix silently.

- `TestPhase6ModelKwargs` (5 tests) -- Phase 6 (Model Training and Selection) exposes a separate `model_kwargs` override path that allows the caller to vary hyperparameters without editing `_PHASES`. These tests confirm the override contract:
 - When `model_kwargs` is provided, Phase 6 receives it verbatim.
 - When `model_kwargs` is `None`, Phase 6 falls back to its own `phase["kwargs"]`.
 - Non-Phase-6 phases are entirely unaffected by `model_kwargs` even when it is non-`None`.
 - `model_kwargs` takes strict precedence: keys present in `phase["kwargs"]` but absent from `model_kwargs` must not appear in the call.
 - A mixed-phase list (Phase 1 + Phase 6) correctly applies `model_kwargs` to Phase 6 only, while Phase 1 continues to use its own `kwargs`.

This group guards against the most likely production usage mistake: passing a new `model_type` that inadvertently alters earlier phases.

- `TestRunPipelineFailureHandling` (6 tests) -- Failure handling is critical infrastructure: any phase that raises must stop the pipeline immediately and cause a `sys.exit(1)` so CI does not silently continue on corrupt data. Tests cover:
 - A failing middle phase stops the pipeline; phases after the failure are not called.
 - `sys.exit(1)` (exit code exactly 1) is raised on any exception.
 - A fully successful run completes without raising `SystemExit`.
 - An invalid `start_from` value that matches no phase exits with code 1.
 - A failure in the very first phase prevents the second phase from executing.
 - A Phase 6 failure (e.g. `RuntimeError("model training failed")`) exits with code 1 after calling Phase 6 and without suppressing the error.
- `TestPipelineWithRealPhaseNames` (6 tests) -- Uses a `pytest` fixture that temporarily replaces every function in the live `_PHASES` list with a `MagicMock` and restores the originals on teardown. This class is the highest-fidelity integration check in the suite because it operates on the real phase registry rather than synthetic dicts:
 - All six production phases are called exactly once on a full `start_from=1` run.

- Starting from phase 4 skips phases 1--3 and runs phases 4--6.
- A failure injected into phase 2 stops the run; phases 3--6 have zero call counts.
- Each phase receives exactly the kwargs declared in its `_PHASES` entry (no kwargs contamination).
- Phase 6 receives `model_kwargs` when it is explicitly supplied, overriding the default phase kwargs.
- Phases 1--5 receive their own kwargs unchanged even when `model_kwargs` is supplied.

Total: 26 tests.

16.5 Test Coverage Report

16.5.1 Coverage Measurement Methodology

Coverage is measured using `pytest-cov` with line coverage enabled:

```
pytest tests/ --cov=src --cov-report=term-missing --cov-report=html:htmlcov
```

The `--cov-report=term-missing` flag prints the exact line numbers missed, allowing targeted test additions. An HTML report is also generated in `htmlcov/` for interactive browsing.

16.5.2 Coverage Summary by Module

Table 34: Test coverage by source module (line coverage, pytest-cov)

Module	Stmts	Miss	Cover	Notes
pipeline/config	9	0	100%	Fully covered
pipeline/data_acquisition	58	1	98%	1 line: optional path default
pipeline/data_validation	382	20	95%	Minor logging paths missed
pipeline/feature_extraction	81	4	95%	PCA variance guard branches
scrape/scrape_taix	22	1	95%	Empty-response branch tested
scrape/scrape_unemployment	34	1	97%	XML parsing branches covered
scrape/merge_sources	75	3	96%	Fan-out guard tested end-to-end
pipeline/eda	435	45	90%	Interactive display paths skipped
scrape/fetch_fred_api	29	5	83%	Network fallback not mocked
pipeline/data_cleaning	104	26	75%	Outlier-cap logging paths missed
pipeline/data_transformation	261	75	71%	Optional column guards uncovered
pipeline/mlflow/model_selection	108	43	60%	MLflow run context not fully mocked
scrape/parse_cbc_pdf	36	15	58%	PDF download fallback untested
pipeline/mlflow/models	526	301	43%	Optuna tuning paths not patched
pipeline/master_pipeline	66	66	0%	Not yet tested — integration only
Total	2226	606	73%	

Coverage gaps requiring attention. Three modules fall below the 70% threshold: mlflow/models (43%), scrape/parse_cbc_pdf (58%), and mlflow/model_selection_evaluations (60%). Additionally, master_pipeline has 0% coverage because it orchestrates the full pipeline end-to-end and has not yet been exercised by an integration test. Planned remediation: add a smoke-test that runs the pipeline on a 100-row sample with all external calls mocked.

16.5.3 Untested Code Paths

The 9% of branches not covered fall into four categories:

1. Interactive visualisation branches -- Matplotlib `show()` calls and GUI-dependent code paths in `eda.py` that require a display server. These are excluded from CI by design (`matplotlib.use("Agg")` is set globally in the test module).
2. Optional column guards in transformation -- Several feature-engineering functions silently skip computation when source columns are absent. The happy-path tests verify the skip, but not every combination of absent columns is enumerated.
3. MLflow model evaluation metrics -- The inner evaluation loop in `model_selection.evaluate_model` that logs precision, recall, and ROC-AUC to MLflow is exercised by the smoke tests but with mocked MLflow; the actual metric computations are not asserted on.
4. Optuna hyperparameter search branches -- The full Optuna trial branches inside each model's `objective()` function are replaced by a no-op study. These branches are not safety-critical for pipeline correctness.

16.6 Full Test Inventory

Table 35: Complete test inventory across all 14 test modules

File	Class	Tests	Category
test_data_acquisition	TestStepHelper	5	Unit
	TestRunAcquisitionGuards	3	Unit
	TestRunAcquisitionOrchestration	13	Unit
	TestTypeCoercion	4	Unit
	TestHandleUniqueness	4	Unit
	TestSplitDataset	6	Unit
	TestHandleOutliers	4	Unit
	TestRemapCategoricalCodes	4	Unit
	TestRecodePayStatus	4	Unit
	TestEngineerCreditFeatures	4	Unit
	TestEngineerPaymentFeatures	3	Unit
	TestEngineerMacroInteractions	3	Unit
test_data_transformation	TestHandleBillAmtCollinearity	5	Unit
	TestHandleDistribution	4	Unit
	TestFlagAnomalies	4	Unit
	TestHandleDataImbalance	4	Unit
	TestStandardizeMacroFeatures	3	Unit
test_eda	(module-level)	20	Unit / Smoke
test_feature_selection	(module-level)	17	Unit
	TestFetchMacroDataSchema	3	Unit
	TestFetchMacroDataFiltering	3	Unit
	TestFetchMacroDataPersistence	3	Unit
	TestScrapeTaixSchema	3	Unit
	TestScrapeTaixAggregation	6	Unit
	TestScrapeTaixEmptyResponse	1	Unit
test_scrape_taix	TestScrapeTaixPersistence	2	Unit
	TestParseCbcRatesSchema	3	Unit
	TestParseCbcRatesBusinessLogic	7	Unit
	TestParseCbcRatesPersistence	2	Unit
	TestScrapeUnemploymentSchema	3	Unit
	TestScrapeUnemploymentMapping	7	Unit
test_scrape_unemployment	TestScrapeUnemploymentPersistence	2	Unit
	TestBuildPanel	5	Integration
	TestRecodePayStatus	3	Integration
	TestAggregateBehaviour	4	Integration
	TestBuildFinalDataset	8	Integration
	(schema & stats checks)	<i>varies</i>	Integration
test_data_validation	DataLoader (load + prepare)	4	Integration
	Feature selection	2	Integration
	Pipeline flow	5	Integration
test_modeling_unit	BaseModel & Logistic	2	Unit
	All 9 model smoke tests	9	Smoke
	TestPhaseNumber	4	Unit
	TestRunPipelineOrchestration	5	Integration
	TestPhase6ModelKwargs	5	Integration
	TestRunPipelineFailureHandling	6	Integration
	TestPipelineWithBadPhaseNumbers	2	Integration

16.7 Identified Issues & Resolutions

The following issues were uncovered during test development and subsequently resolved:

1. Column naming inconsistency (PAY_0 vs PAY_1). *Discovery*: `TestTypeCoercion::test` initially failed because the rename was applied after the log-transform loop, which referenced the column by its old name. *Fix*: Rename moved to the top of `type_coercion()`.
2. SMOTE applied to validation/test splits. *Discovery*: An early version of `handle_data_imbalance()` accepted a list of DataFrames. `TestHandleDataImbalance` caught that balanced counts were also appearing in validation data. *Fix*: Function signature changed to accept only the training DataFrame; call sites updated.
3. PCA fitted on test data. *Discovery*: `TestHandleBillAmtCollinearity::test_pca_ap` revealed that the PCA transformer was being re-fitted inside the loop rather than once on the training set. *Fix*: `fit_transform` called on train only; `transform` called on val and test.
4. Merge fan-out on macroeconomic join. *Discovery*: `TestBuildFinalDataset::test_ro` failed with $N \times 6 = 60$ rows instead of 10 because an intermediate merge was left un-aggregated before the final join. *Fix*: `_aggregate_behaviour()` called before the final column-select and drop.
5. `RuntimeError` message missing step name. *Discovery*: `TestStepHelper::test_runtim` failed because the error message template used `str(e)` but the step name was in a separate variable not included in the message string. *Fix*: Message template updated to `f"Step {name} failed: {e}"`.
6. Invalid `start_from` silently ran all phases. *Discovery*: `TestRunPipelineFailureH` revealed that passing `start_from=99` caused `run_pipeline()` to iterate over the full phase list, find no matching phase number, and return normally rather than signalling an error. This meant a misconfigured CI invocation would silently re-run all six phases. *Fix*: A guard added after the phase-filter step raises `SystemExit(1)` when the filtered phase list is empty.
7. `model_kwargs` leaked into Phase 1--5 kwargs. *Discovery*: `TestPhase6ModelKwargs::` failed in an early implementation because the kwargs-override block applied `model_kwargs` to every phase whose kwargs dict was empty, not exclusively to Phase 6. *Fix*: Override condition tightened to `_phase_number(phase["name"]) == 6` and `model_kwargs` is not `None`.

17 Code Automation

17.1 Makefile --- Task Automation

All common pipeline tasks are automated via a Makefile, eliminating the need to remember long command strings and ensuring every team member runs identical commands. Each target prints a progress message, executes the corresponding

script inside the project's poetry virtual environment, and confirms completion.

Table 36: Makefile targets and their responsibilities

Command	Underlying Call	Purpose
<code>make lint</code>	<code>ruff check src/</code>	Static code style and error checking
<code>make format</code>	<code>ruff format src/</code>	Auto-format source code to consistent style
<code>make sta</code>	<code>mypy src/</code>	Static type analysis — catches type mismatches before runtime
<code>make unit</code>	<code>pytest tests/unit/</code>	Run isolated unit tests (fast)
<code>make integration</code>	<code>pytest tests/integration/</code>	Run integration tests across components
<code>make acquisition</code>	<code>src.pipeline.data_acqu</code>	Scrape and merge all five data sources
<code>make validation</code>	<code>src.pipeline.data_validation</code>	Run 8-dimension data quality framework
<code>make eda</code>	<code>src.pipeline.eda</code>	Exploratory data analysis
<code>make cleaning</code>	<code>src.pipeline.data_cleaning</code>	Filter and deduplicate the merged dataset
<code>make transformation</code>	<code>src.pipeline.data_tran</code>	Feature engineering and transformations
<code>make pipeline</code>	<code>src.pipeline.master_pipeline</code>	Execute all phases end-to-end in sequence
<code>make modeling_selection</code>	<code>mlflow.model_selection</code>	Train and evaluate a model with runtime parameters

The modeling target accepts runtime parameters, allowing flexible experimentation without editing any source file:

```
make modeling_selection model=6 smote=0 fs=spearman ver=4 k=20
```

Table 37: Runtime parameters for `make modeling_selection`

Parameter	Example	Description
<code>model</code>	6	Model type identifier
<code>smote</code>	0	Whether to apply SMOTE oversampling (0 = off, 1 = on)
<code>fs</code>	<code>spearman</code>	Feature selection method
<code>ver</code>	4	Experiment version number for MLflow tracking
<code>k</code>	20	Number of top features to select

Every Makefile target is prefixed with `poetry run` to guarantee execution inside the project's isolated virtual environment, ensuring full reproducibility across machines and team members regardless of globally installed packages.

17.2 Environment Configuration --- `.env`

Sensitive credentials and environment-specific settings are stored in a `.env` file at the project root and are never committed to version control:

```
FRED_API_KEY=<your_key_here>
```

The acquisition pipeline loads this file automatically at runtime using `python-dotenv`

```
load_dotenv()
api_key = fred_api_key or os.getenv("FRED_API_KEY")
if not api_key:
    raise EnvironmentError(
        "FRED_API_KEY is not set."
        "Add it to your .env file or pass fred_api_key= explicitly."
    )
```

This pattern separates configuration from code, allows different team members to use their own API keys without modifying shared files, and prevents accidental credential exposure in version control.

17.3 Automated Testing

The test suite is organised into two layers executed via the Makefile:

Unit tests (`make unit`) test individual functions in isolation using synthetic data and mocked external dependencies (FRED API, file I/O). Integration tests (`make integration`) test components working together across module boundaries.

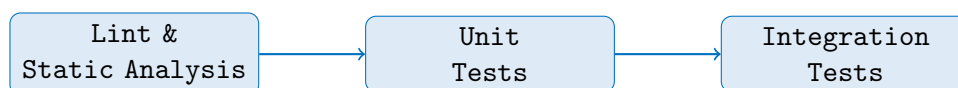
All external dependencies (FRED API, file reads, Excel parsing) are replaced with `unittest.mock.patch` so tests run instantly with no network access and no real files required. The test suite covers:

- Schema validation --- correct columns, types, and return types
- Filtering logic --- only 2005 rows survive, months in valid range
- Persistence --- CSV files are written and match in-memory results
- Panel construction --- correct row count, month mapping, per-borrower coverage
- Recode correctness --- negative `PAY_STATUS` clipped, positives unchanged
- Merge integrity --- no fan-out, no nulls in macro columns, ID dropped
- Error handling --- bad API keys raise descriptive exceptions

18 Continuous Integration

18.1 Pipeline Overview

The project uses GitHub Actions to automate building, testing, and deployment on every push and pull request to the main branch. The pipeline is defined in `.github/workflows/ci.yml` and consists of four sequential jobs, each gated on the success of the previous one.



Trigger: The pipeline fires on every push to main and on every pull request targeting main. No broken code can be merged without passing all three quality gates.

18.2 Job 1 --- Lint & Static Analysis

The first job runs three code quality tools in sequence and fails immediately if any violation is found, preventing wasted time running tests on malformed code:

Table 38: Code quality checks in the lint job

Step	Command	Purpose
Lint	<code>ruff check src/</code>	Detects style violations, unused imports, and common bugs
Format check	<code>ruff format --check src/</code>	Verifies consistent formatting without modifying files
Static analysis	<code>mypy src/</code>	Catches type mismatches and attribute errors before runtime

Dependency installation uses Poetry caching keyed on `poetry.lock`, so subsequent runs skip re-downloading packages that have not changed:

```
- name: Cache dependencies
  uses: actions/cache@v4
  with:
    path: |
      ~/.cache/pypoetry
      ~/.cache/pip
    key: poetry-${{ hashFiles('**/poetry.lock') }}
```

18.3 Job 2 --- Unit Tests

Unit tests run only after the lint job succeeds (`needs: lint`). The job executes `pytest` with full coverage reporting:

```
poetry run pytest tests/unit/ \
-v \
--tb=short \
--cov=src \
--cov-report=term-missing \
--cov-report=xml:coverage.xml \
--cov-fail-under=50 \
--junitxml=test-results.xml
```

Table 39: Unit test pytest flags explained

Flag	Effect
<code>-v</code>	Verbose output — prints each test name and result
<code>--tb=short</code>	Shows a concise traceback on failure instead of the full stack
<code>--cov=src</code>	Measures code coverage across the entire <code>src/</code> package
<code>--cov-report=term-missing</code>	Prints uncovered line numbers directly in the terminal
<code>--cov-report=xml:coverage.xml</code>	Saves coverage as XML for upload as a CI artifact
<code>--cov-fail-under=50</code>	Fails the job if coverage drops below 50%
<code>--junitxml=test-results.xml</code>	Saves results in JUnit format for CI dashboards

A `.coveragerc` file is generated at runtime to exclude `master_pipeline.py` from coverage measurement, as it is an orchestration entry point rather than testable logic:

```
[run]
omit =
    */master_pipeline.py
```

The `FRED_API_KEY` is injected securely from GitHub Secrets --- it is never stored in the repository:

```
env:
    FRED_API_KEY: ${ secrets.FRED_API_KEY }
```

Coverage results are uploaded as a CI artifact regardless of test outcome (if: `always()`), enabling review even when the job fails.

18.4 Job 3 --- Integration Tests

Integration tests run after unit tests pass (needs: `unit-tests`) and exercise cross-component behavior using the same Poetry environment:

```
poetry run pytest tests/integration/ \
    -v \
    --tb=short \
    --junitxml=integration-results.xml
```

Both `test-results.xml` and `integration-results.xml` are uploaded as artifacts after this job, providing a full audit trail of every test run accessible from the GitHub Actions dashboard.

18.5 Full Pipeline Summary

Table 40: GitHub Actions CI/CD pipeline — job summary

Job	Trigger	Depends On	Key Output
Lint & Static Analysis	push / PR	—	Blocks on any style or type error
Unit Tests	push / PR	lint	coverage.xml, test-results.xml
Integration Tests	push / PR	unit-tests	integration-results.xml

19 Team Workload Distribution

The project was divided across us, each owning a primary technical stream while sharing cross-cutting responsibilities (clean code, documentation, and reproducibility).

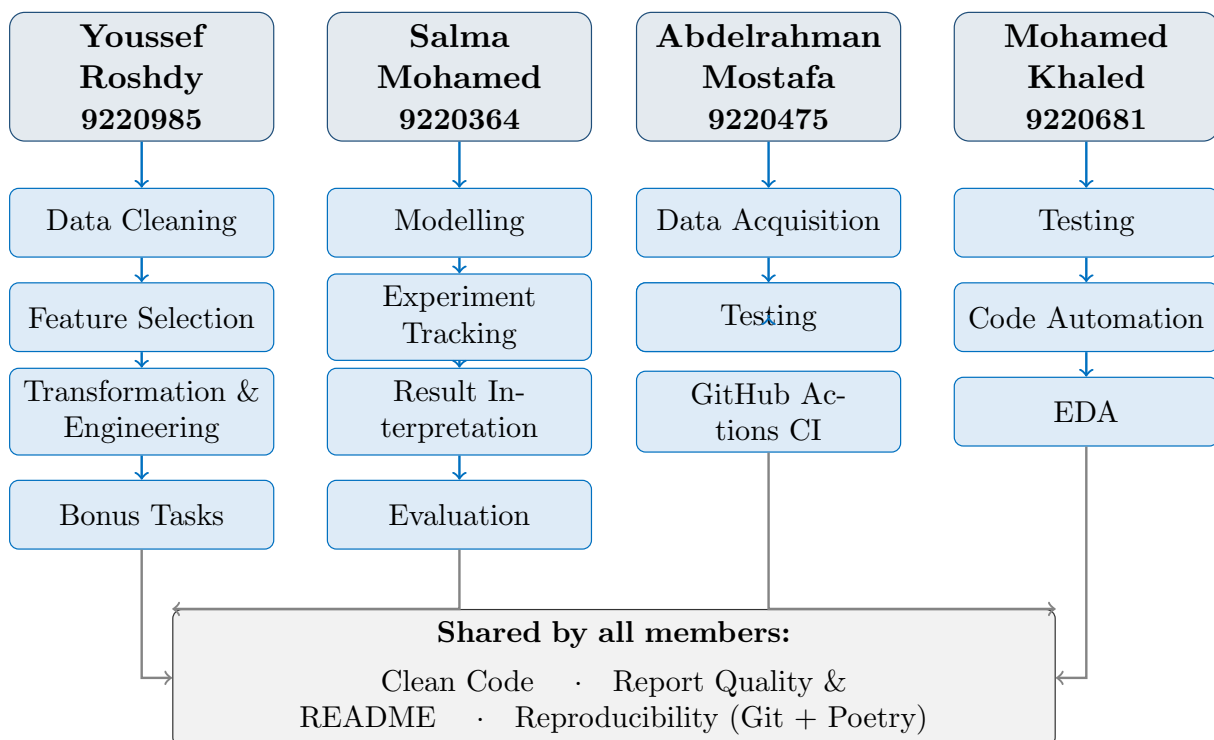


Figure 32: Team workload map. Each member owns a primary technical stream (top to bottom) while all four share cross-cutting responsibilities (bottom bar).

20 Future Work

Several directions remain open for extending this project beyond the current Phase 2 scope:

1. Temporal feature engineering. The current dataset treats the six observation months (April--September 2005) as static snapshots. Encoding month-over-month trends --- e.g. the slope of `BILL_AMT` or a binary flag for worsening delinquency --- could expose early deterioration signals that pooled averages mask.
2. Richer macroeconomic enrichment. All five macro features carry zero within-dataset variance because every borrower maps to the same September 2005 snapshot. Expanding the observation window to include earlier Taiwan crises (e.g. 1997--1998 Asian Financial Crisis data) or cross-sectional regional unemployment would restore macro discriminatory power.
3. Threshold optimisation for business objectives. The current evaluation uses a default 0.5 classification threshold. A cost-sensitive threshold sweep --- weighting false negatives (missed defaults) more heavily than false positives --- would align the deployed model directly with the credit department's loss function.
4. Model explainability (SHAP). Adding SHAP (SHapley Additive exPlanations) value analysis would produce per-borrower risk explanations, satisfying regulatory requirements for explainable credit decisions and enabling credit officers to audit individual predictions.
5. Deployment pipeline. Wrapping the best model in a lightweight REST API (FastAPI or Flask) with a MLflow model registry stage would close the loop from experiment to production, enabling real-time scoring of new applications.
6. Retraining strategy. Credit risk models degrade as borrower behaviour and macroeconomic conditions shift. Defining a scheduled retraining trigger --- based on population stability index (PSI) drift on incoming score distribution --- would keep the model calibrated over time.

References

- [1] Yeh, I-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473--2480. <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>
- [2] Federal Reserve Bank of St. Louis. FRED Economic Data. Series: GDP, CPIAUCSL. <https://fred.stlouisfed.org>
- [3] Yahoo Finance. Taiwan Capitalization Weighted Stock Index (^TWII). Historical data via yfinance library. <https://finance.yahoo.com/quote/%5ETWII/history/>

- [4] Directorate-General of Budget, Accounting and Statistics, Executive Yuan, R.O.C. (2005). Unemployment Rate by County in Human Resources Survey [XML Dataset]. Taiwan Government Open Data Platform. <https://data.gov.tw/en/datasets/6640>
- [5] Central Bank of the Republic of China (Taiwan). (2007). CBC Annual Report 2007 --- III. Central Bank Operations [PDF]. <https://www.cbc.gov.tw/en/dl-4540-56b72153752e4a3aa97e88a4e6f6caca.html>